

Contents

Base64 関数リファレンス	5
関数一覧	5
使用例	5
エラーハンドリング	6
文字列操作関数リファレンス	6
関数一覧	7
contains	8
starts_with	8
ends_with	8
find	8
substring	8
char_at	9
replace	9
to_upper	9
to_lower	9
trim	9
trim_start	10
trim_end	10
repeat	10
reverse	10
byte_len	10
split	10
join	11
to_int	11
to_float	11
to_str	11
組み込み関数リファレンス	12
関数一覧	12
print	14
Some	14
length	15
has_key	15
add	15
remove	15
range	16
exit	16
args	16
sleep	17
env	17
append	18
pop	18
reverse (list)	18
slice	18
take	18
tap	19
filter	19
map	19
sort	19
sort!	20
reverse!	20
appended	20

append!	20
first	20
last	21
get (Map)	21
iter	21
next	21
to_list	22
コレクションリファレンス (タプル・リスト・マップ・セット)	22
タプル	22
リスト	23
マップ	28
セット	30
イテレータ	32
契約による設計 (Design by Contract)	33
事前条件 (require)	33
事後条件 (ensure)	33
組み合わせ例	34
構造体の不変条件 (invariant)	34
ルール	35
制御構文リファレンス	35
if / elif / else	35
while	36
for	36
async / await	38
@parallel for	38
break	38
continue	39
... (Ellipsis)	39
match	39
スコープルール	42
ディレクティブ	42
構文	42
対象	43
組み込みディレクティブ	43
注意事項	47
エラーリファレンス	47
エラーフォーマット	47
コンパイルエラー一覧	47
ランタイムエラー一覧	49
関数リファレンス	49
関数定義の構文	49
引数と戻り値の型	50
再帰	51
オーバーロード	51
デフォルト引数	52
Unit 型関数	53
Task と async 関数	53
ラムダ関数	53
クロージャ	54

関数型	54
高階関数	55
ジェネリック関数	55
UFCS（統一関数呼び出し構文）	56
演算子オーバーロード	57
チェック付き/飽和演算	58
HTTP リファレンス	59
型	59
関数（ <code>http</code> パッケージ）	59
使用例	60
動作仕様	62
サポートされるステータスコード	63
HTTP クライアント	64
エラー処理	66
I/O 関数リファレンス	66
関数一覧	66
使用例	67
エラーハンドリング	68
備考	68
JSON 関数リファレンス	68
概要	68
関数一覧	69
使用例	70
注意事項	71
数学関数 (<code>math</code>)	71
概要	71
定数	71
絶対値	71
丸め	72
冪乗・平方根	72
対数	72
指数	72
三角関数	72
逆三角関数	73
2 引数関数	73
判定関数	73
ネットワーク（TCP）リファレンス	73
型	73
関数（ <code>net</code> パッケージ）	74
組み込みオーバーロード関数	74
使用例	75
タイムアウト設定	76
エラー処理	77
バイト変換	77
演算子リファレンス	77
優先順位テーブル	77
算術演算子	78
比較演算子	78
論理演算子	78

ビット演算子	79
エラー伝播演算子 (? / !!)	79
三項条件演算子	80
範囲演算子	80
null 合体演算子 (??)	80
複合代入演算子	80
インクリメント・デクリメント演算子	81
演算の型規則	81
演算子オーバーロード	82
パッケージリファレンス	85
概要	85
インポート構文	85
パッケージ解決	85
標準ライブラリ (std)	86
検索パスの優先順位	88
RY_PATH 環境変数	88
制約	88
パッケージファイルの作成	88
プロジェクト管理	89
CLI 概要	89
ry init - プロジェクト初期化	89
ry new - 新規プロジェクト作成	90
ry fmt - コードフォーマッター	90
ry test - テスト実行	91
ry self-update - セルフアップデート	91
package.toml 設定ファイル	92
正規表現関数リファレンス	92
関数一覧	93
サポートするパターン構文	93
使用例	94
構造体リファレンス	95
概要	95
定義構文	96
コンストラクタ	96
フィールドアクセス	96
フィールド代入	96
関数引数・戻り値としての使用	96
ネスト構造体	97
比較 (== / !=)	97
レコードのサブタイプ化 (継承)	97
制約とエラー	98
列挙型 (enum)	99
テスト機能	101
実行方法	101
構文	101
出力形式	103
例	103
モック	103
パラメタライズドテスト (@each)	104
プロパティベーステスト (@property)	104

テストカバレッジ	105
制限事項	105
型リファレンス	105
型一覧	105
型アノテーション構文	107
使用可能な型名一覧	107
型エイリアス	108
リテラル型	108
範囲型	108
<code>none</code> キーワードと <code>Option</code> 型の省略記法	109
<code>F-String</code> (文字列補間)	109
型キャスト (<code>as</code>)	110
関連データを持つ <code>enum</code> (ADT)	110
ジェネリック <code>enum</code>	111
<code>Error</code> 型	112
<code>union</code> 型	113
<code>any</code> 型	113
型規則 (演算時の型変換)	115
型安全性の制約	117

[English](#) | [日本語](#) | [繁體中文](#)

Base64 関数リファレンス

Base64 エンコード・デコード。すべての関数は `base64` からの明示的なインポートが必要です。

```
from base64 import encode, decode, encode_url_safe, decode_url_safe
```

関数一覧

関数	シグネチャ	説明
<code>encode</code>	<code>(str) -> str</code>	文字列を標準 base64 にエンコード
<code>decode</code>	<code>(str) -> Result<str, Error></code>	標準 base64 文字列をデコード
<code>encode_url_safe</code>	<code>(str) -> str</code>	文字列を URL-safe base64 にエンコード (パディングなし)
<code>decode_url_safe</code>	<code>(str) -> Result<str, Error></code>	URL-safe base64 文字列をデコード

使用例

基本的なエンコード・デコード

```
from base64 import encode, decode

encoded = encode("Hello, World!")
print(encoded)  # SGVsbG8sIFdvcmkIQ==

match decode(encoded):
    case Ok(s):
        print(s)  # Hello, World!
```

```
case Err(e):
    print(e.message)
```

URL-safe Base64

URL-safe base64 は + と / の代わりに - と _ を使用し、パディング (=) を省略します。URL、ファイル名、トークンに適しています。

```
from base64 import encode_url_safe, decode_url_safe

encoded = encode_url_safe("data with special chars: ?&=")
# 出力に + / = は含まれない

match decode_url_safe(encoded):
    case Ok(s):
        print(s)
    case Err(e):
        print(e.message)
```

バイトデータの操作

バイトデータのエンコード・デコードには io の str_to_bytes / bytes_to_str と組み合わせます。

```
from base64 import encode, decode
from io import str_to_bytes, bytes_to_str

bytes = str_to_bytes("binary data")
encoded = encode(bytes_to_str(bytes))
```

エラーハンドリング

decode と decode_url_safe は Result<str, Error> を返します。無効な base64 文字が含まれている場合、デコードは失敗します。

```
match decode("!!!not-valid!!!"):
    case Ok(s):
        print(s)
    case Err(e):
        print(e.message) # "invalid base64 character at position 0"
```

? 演算子との組み合わせ:

```
fn process(input: str) -> Result<str, Error>:
    decoded = decode(input)?
    return Ok(decoded)
```

[English](#) | [日本語](#) | [繁體中文](#)

文字列操作関数リファレンス

文字列 (str) に対する操作関数の一覧です。すべての関数で UFCS 記法が使用可能です。

注意: すべての文字列操作は UTF-8 対応です。length()、char_at()、substring()、find()、reverse() は Unicode コードポイント単位で動作し、バイト単位ではありません。バイト長が必要な場合は byte_len() を使用してください。

関数一覧

検索・判定

関数	シグネチャ	説明
contains	(str, str) -> bool	部分文字列が含まれるかを返す
starts_with	(str, str) -> bool	接頭辞で始まるかを返す
ends_with	(str, str) -> bool	接尾辞で終わるかを返す
find	(str, str) -> Option<int>	部分文字列の文字位置を返す（未発見は None）

抽出・変換

関数	シグネチャ	説明
substring	(str, int, int) -> str	部分文字列を取得（文字インデックス）
char_at	(str, int) -> str	指定位置の UTF-8 文字を取得
replace	(str, str, str) -> str	部分文字列を全置換

大文字・小文字

関数	シグネチャ	説明
to_upper	str -> str	ASCII 大文字に変換
to_lower	str -> str	ASCII 小文字に変換

空白除去

関数	シグネチャ	説明
trim	str -> str	前後の空白を除去
trim_start	str -> str	先頭の空白を除去
trim_end	str -> str	末尾の空白を除去

生成・加工

関数	シグネチャ	説明
repeat	(str, int) -> str	文字列を n 回繰り返す
reverse	str -> str	文字列を逆順にする（UTF-8 対応）
byte_len	str -> int	文字列のバイト長を返す

分割・結合

関数	シグネチャ	説明
split	(str, str) -> List<str>	デリミタで分割
join	(List<str>, str) -> str	セパレータで結合

型変換

関数	シグネチャ	説明
<code>to_int</code>	<code>str -> int</code>	文字列を整数に変換
<code>to_float</code>	<code>str -> float</code>	文字列を浮動小数点数に変換
<code>to_str</code>	<code>int/float/bool/str/record -> str</code>	値を文字列に変換

contains

シグネチャ: `contains(string: str, substring: str) -> bool`

文字列 `string` に部分文字列 `substring` が含まれるかを返します。

```
print(contains("hello", "ell"))    # true
print("hello".contains("xyz"))     # false (UFCS)
```

starts_with

シグネチャ: `starts_with(string: str, prefix: str) -> bool`

文字列 `string` が `prefix` で始まるかを返します。

```
print(starts_with("hello", "hel"))  # true
print("hello".starts_with("world")) # false (UFCS)
```

ends_with

シグネチャ: `ends_with(string: str, suffix: str) -> bool`

文字列 `string` が `suffix` で終わるかを返します。

```
print(ends_with("hello", "llo"))    # true
print("hello".ends_with("world"))   # false (UFCS)
```

find

シグネチャ: `find(string: str, substring: str) -> Option<int>`

文字列 `string` 中の部分文字列 `substring` の最初の出現位置（文字位置）を返します。見つからない場合は `None` を返します。

```
print(find("hello world", "world")) # Some(6)
print(find("hello", "xyz"))         # None
print("abcdef".find("cd"))          # Some(2) (UFCS)
```

substring

シグネチャ: `substring(string: str, start: int, end: int) -> str`

文字列 `string` の `start` から `end`（排他）までの部分文字列を返します。インデックスは文字位置（UTF-8 対応）です。


```
print(substring("hello world", 0, 5))    # hello
print(substring("hello world", 6, 11))   # world
print("abcdef".substring(1, 4))           # bcd (UFCS)
```

char_at

シグネチャ: `char_at(string: str, i: int) -> str`

文字列 `string` の `i` 番目の UTF-8 文字を文字列として返します。

```
print(char_at("hello", 0))    # h
print("abc".char_at(2))       # c (UFCS)
```

replace

シグネチャ: `replace(string: str, old: str, new: str) -> str`

文字列 `string` 中の `old` をすべて `new` に置換した新しい文字列を返します。

```
print(replace("hello world", "world", "ry"))    # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))       # baz bar baz (UFCS)
```

to_upper

シグネチャ: `to_upper(string: str) -> str`

ASCII 小文字 (a-z) を大文字に変換した新しい文字列を返します。

```
print(to_upper("hello"))    # HELLO
print("Hello World".to_upper()) # HELLO WORLD (UFCS)
```

to_lower

シグネチャ: `to_lower(string: str) -> str`

ASCII 大文字 (A-Z) を小文字に変換した新しい文字列を返します。

```
print(to_lower("HELLO"))    # hello
print("Hello World".to_lower()) # hello world (UFCS)
```

trim

シグネチャ: `trim(string: str) -> str`

文字列の前後の空白文字 (スペース、タブ、改行、復帰) を除去した新しい文字列を返します。

```
print(trim("  hello "))    # hello
print("  hi  ".trim())     # hi (UFCS)
```

trim_start

シグネチャ: `trim_start(string: str) -> str`

文字列の先頭の空白文字を除去した新しい文字列を返します。

```
print(trim_start(" hello "))    # hello
print(" hi".trim_start())       # hi (UFCS)
```

trim_end

シグネチャ: `trim_end(string: str) -> str`

文字列の末尾の空白文字を除去した新しい文字列を返します。

```
print(trim_end(" hello "))    # hello
print("hi ".trim_end())       # hi (UFCS)
```

repeat

シグネチャ: `repeat(string: str, count: int) -> str`

文字列 `string` を `count` 回繰り返した新しい文字列を返します。

```
print(repeat("ab", 3))        # ababab
print("ha".repeat(3))         # hahaha (UFCS)
```

reverse

シグネチャ: `reverse(string: str) -> str`

文字列を逆順にした新しい文字列を返します (UTF-8 対応)。

```
print(reverse("hello"))       # olleh
print("abc".reverse())        # cba (UFCS)
```

byte_len

シグネチャ: `byte_len(string: str) -> int`

文字列 `string` のバイト長を返します。UTF-8 文字数を返す `length()` とは異なり、`byte_len()` はバイト数を返します。

```
print(byte_len("hello"))      # 5
print(byte_len("あいう"))     # 9
print(length("あいう"))       # 3 (文字数)
```

split

シグネチャ: `split(string: str, delimiter: str) -> List<str>`

文字列 `string` をデリミタ `delimiter` で分割し、`List<str>` を返します。

```

parts = split("a,b,c", ",")
print(parts[0])    # a
print(parts[1])    # b
print(parts[2])    # c

for word in "hello world".split(" "):
    print(word)
# hello
# world

```

join

シグネチャ: `join(values: List<str>, sep: str) -> str`

文字列リストの要素をセパレータ `sep` で結合した文字列を返します。

```

parts = ["a", "b", "c"]
print(join(parts, ","))      # a,b,c
print(parts.join("-"))      # a-b-c (UFCS)

```

to_int

シグネチャ: `to_int(string: str) -> int`

文字列を整数に変換します。

```

print(to_int("42"))          # 42
print(to_int("-7"))          # -7
print("123".to_int())        # 123 (UFCS)

```

to_float

シグネチャ: `to_float(string: str) -> float`

文字列を浮動小数点数に変換します。

```

print(to_float("3.14"))      # 3.14
print("2.5".to_float())      # 2.5 (UFCS)

```

to_str

シグネチャ: `to_str(v: int | float | bool | str | record) -> str`

値を文字列に変換します。

型	変換形式
int	%ld
float	%g
bool	"true" / "false"
str	そのまま返す
record	TypeName(field1: val1, field2: val2)

record 型は to_str 表現を自動生成します。ユーザー定義の fn to_str(v: MyRecord) -> str が提供されている場合、自動生成バージョンより優先されます。これは print() や f-string 補間でも同様に機能します。

```
print(to_str(42))          # 42
print(to_str(3.14))        # 3.14
print(to_str(true))        # true
print(99.to_str())         # 99 (UFCS)

record Point:
  x: int
  y: int

p = Point(10, 20)
print(to_str(p))           # Point(x: 10, y: 20)
print(p)                   # Point(x: 10, y: 20)
print(f"pos={p}")          # pos=Point(x: 10, y: 20)
```

[English](#) | [日本語](#) | [繁體中文](#)

組み込み関数リファレンス

関数一覧

コア

関数	説明
print(expr)	値を標準出力に表示
length(value)	リスト・マップ・セットの要素数、文字列の UTF-8 文字数を返す
range(n) / range(start, end) / range(start, end, step)	整数のリストを生成
exit(code)	指定した終了コードでプロセスを終了
args()	コマンドライン引数を List<str> として返す
available_parallelism()	ランタイムの worker 数を int で返す
sleep(duration_ms)	指定ミリ秒間、実行を一時停止する
env(key)	環境変数を Option<str> で返す
env(key, default)	環境変数を返す。未設定なら default を返す
send(stream, data)	TcpStream を通じて List<u8> を送信し、送信バイト数を返す
recv(stream, max)	TcpStream から最大 max バイトを List<u8> として受信
close(handle)	TcpStream または TcpListener を閉じる
block_on(task)	現在のスレッドを Task<T> の完了までブロックし、結果を返す

Option

関数	説明
Some(expr)	Option 型の値ありバリエーションを構築

コレクション操作

関数	説明
has_key(map, key)	マップにキーが存在するかを返す
add(set, value)	セットに要素を追加（重複は無視）
remove(set, value)	セットから要素を削除
append(list, value) / append!(list, value)	リストの末尾に要素を追加（ミューテーション操作）
appended(list, value)	要素を末尾に追加した新しいリストを返す（非破壊）
pop(list)	リストの末尾の要素を削除して Option<T> として返す
reverse(list)	逆順の新しいリストを返す（文字列にも対応）
reverse!(list)	リストをその場で逆順にする（ミューテーション操作）
slice(list, start, end)	start から end までの新しい部分リストを返す
take(list, n)	先頭 n 要素の新しいリストを返す
tap(list, fn)	各要素に fn を呼び出し、元のリストを返す
filter(list, pred)	述語を満たす要素だけの新しいリストを返す
map(list, fn)	各要素を変換した新しいリストを返す
sort(list) / sort(list, comp)	ソート済みの新しいリストを返す（デフォルト昇順）
sort!(list) / sort!(list, comp)	リストをその場でソートする（ミューテーション操作）
insert(list, i, val)	インデックス i に要素を挿入
remove_at(list, i)	インデックス i の要素を削除して返す
items(map)	(キー, 値) タプルのリストを返す
remove(map, key)	指定したキーのエントリを削除
get(map, key)	キーの値を Option<V> として返す
get(map, key, default)	キーの値を返す（存在しない場合はデフォルト値）
union(set, set)	2つのセットの和集合を返す
intersection(set, set)	2つのセットの積集合を返す
difference(set, set)	2つのセットの差集合を返す
symmetric_difference(set, set)	2つのセットの対称差を返す
is_subset(set, set)	最初のセットが2番目の部分集合かを返す
is_superset(set, set)	最初のセットが2番目の上位集合かを返す

イテレータ

関数	説明
iter(collection)	List、Set、Map から遅延イテレータを作成
next(iter)	次の要素を Option<T> として返す。使い切った場合は None
to_list(iter)	イテレータの残りの要素をすべて List<T> に収集
filter(iter, pred)	述語にマッチする要素のみを返す遅延イテレータを返す
map(iter, fn)	各要素を変換する遅延イテレータを返す
take(iter, n)	最大 n 要素を返す遅延イテレータを返す

文字列操作

関数	説明
contains(string, substring)	部分文字列が含まれるか
starts_with(string, prefix)	接頭辞で始まるか
ends_with(string, suffix)	接尾辞で終わるか
find(string, substring)	部分文字列の文字位置 (Option<int>)
byte_len(string)	文字列のバイト長を返す

関数	説明
<code>substring(string, start, end)</code>	部分文字列を取得
<code>char_at(string, i)</code>	指定位置の文字を取得
<code>replace(string, old, new)</code>	部分文字列を全置換
<code>to_upper(string) / to_lower(string)</code>	大文字・小文字変換
<code>trim(string) / trim_start(string) / trim_end(string)</code>	空白除去
<code>repeat(string, count)</code>	文字列を n 回繰り返す
<code>reverse(string)</code>	文字列を逆順にする
<code>split(string, delimiter)</code>	文字列を分割してリストを返す
<code>join(list, sep)</code>	リストの文字列をセパレータで結合
<code>to_int(s) / to_float(s) / to_str(v)</code>	型変換

-> 詳細は[文字列操作関数リファレンス](#)を参照

print

シグネチャ: `print(expr)`

値を標準出力に表示します。末尾に改行が付きます。

型	出力形式
<code>int</code>	<code>%ld</code>
<code>float</code>	<code>%g</code>
<code>bool</code>	<code>true / false</code>
<code>str</code>	<code>%s</code>
<code>Option (Some)</code>	<code>Some(値)</code>
<code>Option (None)</code>	<code>None</code>
<code>list</code>	<code>[要素 1, 要素 2, ...]</code>
<code>map</code>	<code>{キー 1: 値 1, キー 2: 値 2, ...}</code>
<code>set</code>	<code>{要素 1, 要素 2, ...}</code>
<code>enum</code>	バリエーション名 (例: <code>Red</code>)

```

print(42)           # 42
print(3.14)          # 3.14
print(true)          # true
print("hello")       # hello
print(Some(1))       # Some(1)
print(None)          # None
print([1, 2, 3])     # [1, 2, 3]
print({"a": 1})      # {a: 1}
print({1, 2, 3})     # {1, 2, 3}

```

エラー条件: 構造体・タプルを直接渡すとコンパイルエラー。

Some

シグネチャ: `Some(expr) -> Option<T>`

`Option` 型の値ありバリエーションを構築します。

```
x: Option<int> = Some(42)
print(x)      # Some(42)
```

length

シグネチャ: `length(x: List<T> | Map<K, V> | Set<T> | str) -> int`

リスト・マップ・セットの要素数、または文字列の UTF-8 文字数を返します。バイト長が必要な場合は `byte_len()` を使用してください。

```
print(length([1, 2, 3]))      # 3
print(length({"a": 1, "b": 2})) # 2
print(length({1, 2, 3}))      # 3
print(length("hello"))        # 5
print(length("あいう"))       # 3 (UTF-8 文字数)
```

has_key

シグネチャ: `has_key(m: Map<K, V>, key: K) -> bool`

マップに指定したキーが存在するかを返します。UFCS 記法も使用可能です。

```
m = {"a": 1, "b": 2}
print(has_key(m, "a"))      # true
print(m.has_key("z"))       # false (UFCS)
```

add

シグネチャ: `add(s: Set<T>, value: T)`

セットに要素を追加します。既に存在する要素を追加した場合は何もしません。UFCS 記法も使用可能です。

```
s = {1, 2, 3}
s.add(4)      # UFCS
add(s, 5)     # 通常の呼び出し
s.add(1)      # 既に存在するため無視
print(length(s)) # 5
```

remove

シグネチャ: `remove(s: Set<T>, value: T)`

セットから要素を削除します。UFCS 記法も使用可能です。

```
s = {1, 2, 3}
s.remove(2)    # UFCS
print(2 in s)  # false
```

range

シグネチャ: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

整数のリストを生成します。

形式	生成される値
<code>range(n)</code>	<code>[0, 1, ..., n-1]</code>
<code>range(start, end)</code>	<code>[start, start+1, ..., end-1]</code>
<code>range(start, end, step)</code>	<code>[start, start+step, start+2*step, ...]</code> (end は含まない)

- `step > 0` の場合、`start` から昇順に `end` に向かって生成します。
- `step < 0` の場合、`start` から降順に `end` に向かって生成します。
- `step == 0` の場合、ランタイムエラーになります。
- 範囲が空の場合 (例: `range(0, 10, -1)`)、空リストを返します。

```
print(range(3))           # [0, 1, 2]
print(range(2, 5))        # [2, 3, 4]
print(range(0, 10, 2))    # [0, 2, 4, 6, 8]
print(range(10, 0, -3))   # [10, 7, 4, 1]
```

```
for i in range(3):
    print(i)
# 0
# 1
# 2
```

exit

シグネチャ: `exit(code: int)`

指定した終了コードでプロセスを即座に終了します。`exit()` 以降のコードは到達不能になります。

```
exit(0)    # 正常終了
exit(1)    # エラー終了
```

args

シグネチャ: `args() -> List<str>`

スクリプトに渡されたコマンドライン引数を文字列のリストとして返します。インタプリタ名やスクリプトファイル名は含まれません。スクリプトパスの後の引数のみです。

```
# 実行: ry script.ry hello world
a = args()
print(length(a))    # 2
print(a[0])         # hello
print(a[1])         # world

for x in args():
    print(x)
```


sleep

シグネチャ: `sleep(duration_ms: int) -> Unit`

指定されたミリ秒間、現在のスレッドの実行を一時停止します。`duration_ms` が 0 以下の場合は即座に返ります。

```
sleep(1000)    # 1秒待機
sleep(0)       # 即座に戻る
```

env

シグネチャ: `env(key: str) -> Option<str> / env(key: str, default: str) -> str`

環境変数の値を返します。1 引数の場合は `Option<str>` (設定済みなら `Some(value)`、未設定なら `None`) を返します。2 引数の場合は値が未設定なら `default` を返します。

プロジェクトルート (`package.toml` が存在するディレクトリ) に `.env` ファイルがある場合、起動時に自動的に読み込まれます。既存の環境変数は `.env` の値で上書きされません。

セキュリティ上の注意: `.env` ファイルには通常シークレット (API キー、データベースパスワード、トークン等) が含まれます。`.env` をバージョン管理にコミットしないでください (`.gitignore` 等に追加してください)。その内容は機密情報として扱ってください。

```
# 1引数: Option<str> を返す
path = env("PATH")
match path:
  case Some(v):
    print(v)
  case None:
    print("PATH not set")

# 2引数: デフォルト値付き
port = env("PORT", "8080")
print(port)  # PORT 未設定なら "8080"
```

`.env` ファイルの書式

```
# コメントは # で始まる
DATABASE_URL=postgres://localhost/mydb
API_KEY="secret-key-123"
EMPTY_VALUE=
QUOTED='single quoted'
```

環境別 `.env` ファイル

`RY_ENV` が設定されている場合、`Ry` は以下の優先順位で環境別の `.env` ファイルを読み込みます:

- `.env.<環境名>` を最初に読み込む (例: `RY_ENV=dev` なら `.env.dev`)
- `.env` を次に読み込む (`.env.<環境名>` で設定済みの値は上書きされない)
- `RY_ENV=prod` の場合、`.env` ファイルは一切読み込まない (セキュリティ)
- `RY_ENV` が未設定の場合、`.env` のみ読み込む (後方互換)

環境モードの詳細は [RY_ENV](#) を参照してください。

append

シグネチャ: `append(list: List<T>, value: T)`

リストの末尾に要素を追加します。これはミューテーション操作で、リストがその場で変更されます。UFCS 記法も使用可能です。

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

シグネチャ: `pop(list: List<T>) -> Option<T>`

リストの末尾の要素を削除して `Option<T>` として返します。リストが空の場合は `None` を返します。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
v = xs.pop()
print(v)      # Some(3)
print(xs)     # [1, 2]
```

reverse (list)

シグネチャ: `reverse(list: List<T>) -> List<T>`

要素を逆順にした新しいリストを返します。元のリストは変更されません。文字列に対しても使用できます ([文字列操作関数リファレンス](#)を参照)。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
ys = reverse(xs)
print(ys)     # [3, 2, 1]
print(xs)     # [1, 2, 3] (変更なし)
```

slice

シグネチャ: `slice(list: List<T>, start: int, end: int) -> List<T>`

`start` (含む) から `end` (含まない) までの新しい部分リストを返します。インデックスは有効範囲 (0 から `length(list)` まで) にクランプされます。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (クランプされる)
```

take

シグネチャ: `take(list: List<T>, n: int) -> List<T>`

先頭 `n` 要素の新しいリストを返します。`n` がリストの長さを超える場合はリスト全体のコピーを返します。`n <= 0` の場合は空リストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)      # [1, 2, 3]
print(xs.take(10)) # [1, 2, 3, 4, 5] (クランプされる)
print(xs.take(0))  # []
```

tap

シグネチャ: `tap(list: List<T>, fn: fn(T) -> R) -> List<T>`

各要素に対して関数を呼び出し（戻り値は無視）、元のリストをそのまま返します。メソッドチェーン中のデバッグや副作用の挿入に有用です。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
ys = xs.tap(fn(x: int) => print(x)).map(fn(x: int) => x * 2)
# 1, 2, 3 を出力し、ys = [2, 4, 6]
```

filter

シグネチャ: `filter(list: List<T>, pred: fn(T) -> bool) -> List<T>`

述語が `true` を返す要素のみを含む新しいリストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter(fn(x: int) => x > 3)
print(ys)      # [4, 5]
print(xs)      # [1, 2, 3, 4, 5] (変更なし)
```

map

シグネチャ: `map(list: List<T>, fn: fn(T) -> U) -> List<U>`

各要素を関数で変換した新しいリストを返します。出力の要素型は入力と異なっても構いません。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
ys = xs.map(fn(x: int) => x * 2)
print(ys)      # [2, 4, 6]
```

sort

シグネチャ: `sort(list: List<T>) -> List<T> / sort(list: List<T>, comp: fn(T, T) -> bool) -> List<T>`

ソート済みの新しいリストを返します。デフォルトは昇順です。カスタム比較関数を指定できます（第一引数が第二引数の前に来るべき場合に `true` を返す）。元のリストは変更されません。ソートは安定です（等しい要素の元の順序が保持されます）。UFCS 記法も使用可能です。

```
xs = [3, 1, 2]
print(xs.sort()) # [1, 2, 3]
```

降順ソート

```
desc = xs.sort(fn(a: int, b: int) => a > b)
print(desc)    # [3, 2, 1]
```

sort!

シグネチャ: `sort!(list: List<T>) / sort!(list: List<T>, comp: fn(T, T) -> bool)`

リストをその場でソートします。ソートアルゴリズムは `sort()` と同じですが、新しいリストを作成する代わりに元のリストを変更します。UFCS 記法も使用可能です。

```
xs = [3, 1, 2]
xs.sort!()
print(xs)    # [1, 2, 3]
```

reverse!

シグネチャ: `reverse!(list: List<T>)`

リストをその場で逆順にします。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
xs.reverse!()
print(xs)    # [3, 2, 1]
```

appended

シグネチャ: `appended(list: List<T>, value: T) -> List<T>`

要素を末尾に追加した新しいリストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2]
ys = xs.appended(3)
print(xs)    # [1, 2] (変更なし)
print(ys)    # [1, 2, 3]
```

append!

シグネチャ: `append!(list: List<T>, value: T)`

`append()` のエイリアスです。リストの末尾に要素をその場で追加します。! 命名規約との一貫性のために提供されています。

first

シグネチャ: `first(list: List<T>) -> Option<T>`

リストの最初の要素を `Option<T>` として返します。リストが空の場合は `None` を返します。

```
print(first([10, 20, 30]))    # Some(10)
```

last

シグネチャ: `last(list: List<T>) -> Option<T>`

リストの最後の要素を `Option<T>` として返します。リストが空の場合は `None` を返します。

```
print(last([10, 20, 30]))    # Some(30)
```

get (Map)

シグネチャ: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

2 引数形式はキーの値を `Option<V>` として返します。3 引数形式はキーの値またはデフォルト値を返します。

```
m = {"a": 1, "b": 2}
print(get(m, "a"))           # Some(1)
print(get(m, "z"))           # None
print(get(m, "z", 0))        # 0
```

iter

シグネチャ: `iter(collection: List<T> | Set<T>) -> Iterator<T>` / `iter(collection: Map<K, V>) -> Iterator<(K, V)>`

コレクションから遅延イテレータを作成します。イテレータはデータをコピーせず、元のコレクションを参照します。UFCS 記法も使用可能です。

- `List<T>` と `Set<T>` の場合、要素型は `T`。
- `Map<K, V>` の場合、要素型はタプル `(K, V)`。

```
xs = [1, 2, 3]
it = xs.iter()               # Iterator<int>
ys = it.to_list()            # [1, 2, 3]

m = {"a": 1, "b": 2}
for k, v in m.iter():         # Iterator<(str, int)>
    print(k)
```

next

シグネチャ: `next(iter: Iterator<T>) -> Option<T>`

イテレータから次の要素を `Option<T>` として返します。イテレータが使い切られた場合は `None` を返します。呼び出しごとにイテレータの内部状態が進みます。UFCS 記法も使用可能です。

```
it = [10, 20].iter()
print(it.next())             # Some(10)
print(it.next())             # Some(20)
print(it.next())             # None
```

to_list

シグネチャ: `to_list(iter: Iterator<T>) -> List<T>`

イテレータの残りの要素をすべて新しいリストに収集します。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
ys = xs.iter().filter(fn(x: int) => x > 2).to_list()
print(ys)    # [3, 4, 5]
```

[English](#) | [日本語](#) | [繁體中文](#)

コレクションリファレンス (タプル・リスト・マップ・セット)

タプル

概要

固定長・異種型の値の組み合わせ。LLVM literal StructType として実装されたスタック上の値型です。

構文

```
t = (1, 3.14)
t: (int, float) = (1, 3.14)
```

型アノテーション

```
pair: (int, str) = (42, "hello")
triple: (int, float, bool) = (1, 2.0, true)
```

要素アクセス

.0, .1, ... の数値インデックスでアクセスします。

```
t = (10, 3.14)
print(t.0)    # 10
print(t.1)    # 3.14
```

関数戻り値

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0)    # 2
print(result.1)    # 1
```

制約とエラー

制約	詳細
範囲外インデックス	コンパイルエラー
print にタプルを直接渡す	コンパイルエラー (print 非対応)

リスト

概要

同一型の可変長シーケンス。ヒープ上に確保されます。

構文

```
xs = [1, 2, 3]
xs: List<int> = [1, 2, 3]
```

対応する要素型

int, float, bool, str

インデックスアクセス

```
xs = [1, 2, 3]
print(xs[0])    # 1
print(xs[2])    # 3
```

インデックス代入

```
xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
```

length

```
xs = [1, 2, 3]
print(length(xs))    # 3
```

print

```
xs = [1, 2, 3]
print(xs)    # [1, 2, 3]
```

for 走査

```
xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

リストの末尾に要素を追加します。これはミューテーション操作です。

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

リストの末尾の要素を削除して返します。空のリストに対して呼び出すとランタイムエラーになります。

```
xs = [1, 2, 3]
v = xs.pop()
print(v)      # 3
print(xs)     # [1, 2]
```

reverse

要素を逆順にした新しいリストを返します。元のリストは変更されません。文字列に対しても使用できます。

```
xs = [1, 2, 3]
print(reverse(xs)) # [3, 2, 1]
print(xs)          # [1, 2, 3] (変更なし)
```

slice

start (含む) から end (含まない) までの新しい部分リストを返します。インデックスは有効範囲にクランプされます。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3)) # [2, 3]
print(slice(xs, 0, 100)) # [1, 2, 3, 4, 5] (クランプされる)
```

take

先頭 n 要素の新しいリストを返します。n がリストの長さを超える場合はリスト全体のコピーを返します。n <= 0 の場合は空リストを返します。元のリストは変更されません。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys) # [1, 2, 3]
print(xs.take(10)) # [1, 2, 3, 4, 5] (クランプされる)
print(xs.take(0)) # []
```

tap

各要素に対して関数を呼び出し (戻り値は無視)、元のリストをそのまま返します。メソッドチェーン中のデバッグや副作用の挿入に有用です。

```
xs = [1, 2, 3]
ys = xs.tap(fn(x: int) => print(x)).map(fn(x: int) => x * 2)
# 1, 2, 3 を出力し、ys = [2, 4, 6]
```

filter

述語を満たす要素だけを含む新しいリストを返します。元のリストは変更されません。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter(fn(x: int) => x > 3)
print(ys) # [4, 5]
```

map

各要素を関数で変換した新しいリストを返します。出力の要素型は入力と異なっても構いません。元のリストは変更されません。


```
xs = [1, 2, 3]
ys = xs.map(fn(x: int) => x * 2)
print(ys)    # [2, 4, 6]
```

sort

ソート済みの新しいリストを返します。デフォルトは昇順です。カスタム比較関数を指定できます。元のリストは変更されません。ソートは安定です（等しい要素の元の順序が保持されます）。内部的に TimSort を使用しています。

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# 降順ソート
desc = xs.sort(fn(a: int, b: int) => a > b)
print(desc)    # [3, 2, 1]
```

filter, map, sort のチェーン

これらの関数は新しいリストを返すため、UFCS で連鎖できます。

```
xs = [5, 3, 1, 4, 2]
result = xs.filter(fn(x: int) => x > 1).map(fn(x: int) => x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce

アキュムレータ関数を使ってリストを単一の値に畳み込みます。最初の要素を初期値として使用します。

```
xs = [1, 2, 3, 4, 5]
total = reduce(xs, fn(a: int, b: int) => a + b)
print(total)    # 15
```

fold

明示的な初期値とアキュムレータ関数を使ってリストを単一の値に畳み込みます。

```
xs = [1, 2, 3, 4, 5]
total = fold(xs, 0, fn(a: int, b: int) => a + b)
print(total)    # 15
```

any

述語を満たす要素が 1 つ以上あれば true を返します。

```
xs = [1, 2, 3, 4, 5]
print(any(xs, fn(x: int) => x > 4))    # true
print(any(xs, fn(x: int) => x > 9))    # false
```

all

すべての要素が述語を満たす場合に true を返します。

```
xs = [2, 4, 6]
print(all(xs, fn(x: int) => x > 0))    # true
print(all(xs, fn(x: int) => x > 3))    # false
```

sum

全要素の合計を返します。

```
xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

min

最小の要素を返します。

```
xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

max

最大の要素を返します。

```
xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

first

最初の要素を返します。空のリストに対して呼び出すとランタイムエラーになります。

```
xs = [10, 20, 30]
print(first(xs))   # 10
```

last

最後の要素を返します。空のリストに対して呼び出すとランタイムエラーになります。

```
xs = [10, 20, 30]
print(last(xs))    # 30
```

is_empty

リストが空であれば true を返します。

```
xs = [1, 2, 3]
print(is_empty(xs))    # false
print(is_empty([]))    # true (実際には型注釈が必要)
```

enumerate

(インデックス, 要素) のタプルのリストを返します。

```
xs = [10, 20, 30]
pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# for ループでのタプル分解
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

zip

2つのリストを (要素1, 要素2) のタプルのリストに結合します。結果の長さは短い方のリストと同じになります。

```

xs = [1, 2, 3]
ys = ["a", "b", "c"]
pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# for ループでのタプル分解
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c

```

insert

指定したインデックスに要素を挿入します。そのインデックス以降の要素は右にシフトされます。

```

xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]

```

remove_at

指定したインデックスの要素を削除して返します。そのインデックス以降の要素は左にシフトされます。

```

xs = [1, 2, 3, 4]
v = remove_at(xs, 1)
print(v)    # 2
print(xs)   # [1, 3, 4]

```

remove

リストから指定した値の最初の出現を削除します。値が見つからない場合は何もしません。破壊的操作です。

```

xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)    # [1, 3, 2, 4]

```

distinct

重複を排除した新しいリストを返します。元の順序は保持されます（最初の出現を残します）。元のリストは変更されません。

```

xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))    # [1, 2, 3, 4]
print(xs)              # [1, 2, 3, 2, 1, 4] (変更なし)

```

flatten

ネストされたリスト（リストのリスト）を1段階フラット化します。新しいリストを返します。元のリストは変更されません。

```

xs = [[1, 2], [3, 4]]
print(flatten(xs))    # [1, 2, 3, 4]
print(xs)             # [[1, 2], [3, 4]] (変更なし)

```

操作の計算量

操作	計算量
<code>xs[i]</code> インデックスアクセス	$O(1)$
<code>append</code> / <code>append!</code>	$O(1)$ (均償)

操作	計算量
pop	O(1)
first, last	O(1)
insert, remove_at	O(n)
sort / sort!	O(n log n)
take	O(n)
tap	O(n)
filter, map, reduce, fold	O(n)
reverse / reverse!	O(n)
distinct	O(n)
length	O(1)

制約とエラー

制約	詳細
全要素は同一型	異なる型が混在するとコンパイルエラー
空リスト []	型推論できないためコンパイルエラー
範囲外アクセス	ランタイムエラー (exit(1))

マップ

概要

キーと値の対応表。ヒープ上に確保されます。

構文

```
m = {"a": 1, "b": 2}
m: Map<str, int> = {"a": 1, "b": 2}
```

キーアクセス

```
m = {"a": 1, "b": 2}
print(m["a"])    # 1
```

挿入・更新

```
m = {"a": 1}
m["b"] = 2      # 新規追加
m["a"] = 99     # 更新
```

length

```
m = {"a": 1, "b": 2, "c": 3}
print(length(m))    # 3
```

print

```
m = {"a": 1, "b": 2}
print(m)    # {a: 1, b: 2}
```

has_key

```
m = {"a": 1, "b": 2}
print(m.has_key("a"))    # true
print(m.has_key("z"))    # false
```

keys

マップの全キーのリストを返します。

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))           # ["a", "b", "c"]
```

values

マップの全値のリストを返します。

```
m = {"a": 1, "b": 2, "c": 3}
print(values(m))         # [1, 2, 3]
```

items

マップの全エントリの (キー, 値) タプルのリストを返します。

```
m = {"a": 1, "b": 2}
pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (マップ)

指定したキーのエントリをマップから削除します。キーが存在しない場合は何もありません。

```
m = {"a": 1, "b": 2}
remove(m, "a")
print(m)           # {b: 2}
```

get

指定したキーの値を返します。キーが存在しない場合はデフォルト値を返します。

```
m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

merge

2つのマップを結合した新しいマップを返します。キーが重複する場合は第2マップの値が優先されます。元のマップは変更されません。

```
m1 = {"a": 1, "b": 2}
m2 = {"b": 99, "c": 3}
m3 = merge(m1, m2)
print(m3["a"])           # 1
print(m3["b"])           # 99
print(m3["c"])           # 3
```

制約とエラー

制約	詳細
全キーは同一型	異なる型のキーが混在するとコンパイルエラー
全値は同一型	異なる型の値が混在するとコンパイルエラー
空マップ	型注釈が必要 (<code>m: Map<str, int> = {"a": 1}</code> など)
存在しないキーアクセス	ランタイムエラー (<code>exit(1)</code>)
キー検索	ハッシュテーブル (平均 $O(1)$)
容量超過時	自動で 2 倍に拡張

セット

概要

同一型の要素を重複なしで保持するコレクション。ヒープ上に確保されます。

構文

```
s = {1, 2, 3}
s: Set<int> = {1, 2, 3}
```

対応する要素型

`int`, `float`, `bool`, `str`

in 演算子 (所属チェック)

```
s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false
```

length

```
s = {1, 2, 3}
print(length(s)) # 3
```

print

```
s = {1, 2, 3}
print(s)    # {1, 2, 3}
```

add (要素追加)

重複する要素を追加した場合は無視されます。

```
s = {1, 2, 3}
s.add(4)    # 追加
s.add(1)    # 既に存在するため無視
print(length(s))    # 4
```

remove (要素削除)

```
s = {1, 2, 3}
s.remove(2)
print(2 in s)    # false
```

for 走査

```
s = {10, 20, 30}
for x in s:
    print(x)
```

空セット

空セットは型注釈が必要です。

```
s: Set<int> = {}
```

関数引数

```
fn has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

両方のセットの全要素を含む新しいセットを返します。

```
a = {1, 2, 3}
b = {3, 4, 5}
print(union(a, b))    # {1, 2, 3, 4, 5}
```

intersection

両方のセットに存在する要素のみを含む新しいセットを返します。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(intersection(a, b))    # {2, 3}
```

difference

最初のセットにはあるが、2 番目のセットにはない要素を含む新しいセットを返します。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(difference(a, b))    # {1}
```

symmetric_difference

いずれかのセットにあるが、両方にはない要素を含む新しいセットを返します。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(symmetric_difference(a, b))    # {1, 4}
```

is_subset

最初のセットの全要素が 2 番目のセットに含まれている場合に true を返します。

```
a = {1, 2}
b = {1, 2, 3}
print(is_subset(a, b))    # true
print(is_subset(b, a))    # false
```

is_superset

最初のセットが2番目のセットの全要素を含んでいる場合に true を返します。

```
a = {1, 2, 3}
b = {1, 2}
print(is_superset(a, b))    # true
print(is_superset(b, a))    # false
```

制約とエラー

制約	詳細
全要素は同一型	異なる型が混在するとコンパイルエラー
空セット {}	型注釈が必要
要素検索	ハッシュテーブル (平均 $O(1)$)
容量超過時	自動で2倍に拡張

イテレータ

概要

効率的なデータ変換パイプラインを実現する遅延イテレータの抽象化です。イテレータは中間結果のコピーや実体化を行わず、各要素をオンデマンドで処理します。

イテレータの作成

iter() を使って任意のコレクションからイテレータを作成します:

```
xs = [1, 2, 3, 4, 5]
it = xs.iter()           # Iterator<int>

s = {10, 20, 30}
sit = s.iter()           # Iterator<int>

m = {"a": 1, "b": 2}
mit = m.iter()           # Iterator<(str, int)>
```

遅延メソッドチェーン

イテレータのメソッドは新しいイテレータを返し、消費されるまで評価されないパイプラインを形成します:

メソッド	説明
.filter(fn)	述語が true を返す要素のみを生成
.map(fn)	各要素を関数で変換
.take(n)	最大 n 要素を生成

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter(fn(x: int) => x > 2)
    .map(fn(x: int) => x * 2)
    .take(2)
    .to_list()    # [6, 8]
```


イテレータの消費

メソッド	説明
<code>.to_list()</code>	全要素を <code>List<T></code> に収集
<code>.next()</code>	次の要素を <code>Option<T></code> として返す

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

for ループのサポート

イテレータは for ループで直接使用できます:

```
for x in [1, 2, 3].iter().filter(fn(x: int) => x > 1):
    print(x)
# 2
# 3

for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

[English](#) | [日本語](#) | [繁體中文](#)

契約による設計 (Design by Contract)

Ry は Eiffel スタイルの契約による設計をサポートしています。事前条件 (require)、事後条件 (ensure)、構造体の不変条件 (invariant) を使用できます。契約違反時はプロセスが `exit(1)` で終了します。

事前条件 (require)

事前条件は関数の入口でチェックされます。関数が正しく呼び出されるために満たすべき条件を指定します。

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    new_balance: int = balance + amount
    return new_balance
```

事前条件が満たされない場合、以下のメッセージとともにプログラムが終了します:

```
Contract violation: require failed in deposit()
```

事後条件 (ensure)

事後条件はすべての `return` の直前にチェックされます。関数の戻り値について保証する条件を指定します。

変数バインド

`ensure` には戻り値をバインドする変数名が必要です。この変数を事後条件の式で使用できます。

```
fn abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

Ry の関数引数はイミュータブルなので、ensure ブロック内で引数を直接参照して入口時の値と比較できます:

```
fn increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

タプル展開

タプルを返す関数では、カンマ区切りで複数の変数名を指定できます:

```
fn divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

バインド変数の数はタプルの要素数と一致する必要があります。

組み合わせ例

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

構造体の不変条件 (invariant)

不変条件は構造体のインスタンスが常に満たすべき条件です。以下のタイミングでチェックされます: - 構築時
- フィールド代入後

```
record BankAccount:
  balance: int
  min_balance: int
  invariant:
    balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1             # Contract violation: invariant failed
```

ルール

- `require` と `ensure` ブロックはオプションで、関数本体の前に記述します。
- 両方を使う場合、`require` は `ensure` の前に記述する必要があります。
- `ensure` には戻り値をバインドする変数名が必要です (例: `ensure v:`)。
- タプル戻り値の場合、複数のバインド変数を指定できます (例: `ensure q, r:`)。
- `invariant` は `record` 定義の末尾、全フィールド宣言の後に記述します。
- すべての契約違反は `exit(1)` でプログラムを終了し、診断メッセージを出力します。

[English](#) | [日本語](#) | [繁體中文](#)

制御構文リファレンス

if / elif / else

構文

```
if 条件式:
    # then ブロック
elif 条件式:
    # elif ブロック (複数可)
else:
    # else ブロック (省略可)
```

条件式の型

型	false になる値	true になる値
bool	false	true
int	0	非 0

`float` や `str` は条件式に直接使用できない。

例

```
x = 10

if x > 5:
    print("big")
elif x == 5:
    print("five")
else:
    print("small")
```

スコープルール

- `if / elif / else` の各ブロックはそれぞれ独立したブロックスコープを持つ。
- ブロック内で宣言した変数はブロック外からアクセスできない。

```
if true:
    y = 42
# y はここではアクセス不可
```

while

構文

```
while 条件式:  
    # ループ本体
```

条件式が `true` の間、ループ本体を繰り返す。

例

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

break / continue との組み合わせ

```
i = 0  
while true:  
    if i >= 3:  
        break  
    i += 1
```

for

構文

```
# リスト / セット走査  
for x in iterable_expr:  
    # x に各要素が代入される  
  
# range (0 始まり)  
for i in range(n):  
    # i = 0, 1, ..., n-1  
  
# range (開始・終了指定)  
for i in range(start, end):  
    # i = start, start+1, ..., end-1  
  
# range (ステップ指定)  
for i in range(start, end, step):  
    # i = start, start+step, start+2*step, ...
```

マップのキー・値走査

```
for k, v in map_expr:  
    # k はキー、v は各エントリの値
```

タプル分解代入

タプルのリストを走査する際、タプルの要素数に合わせて `N` 個の変数に分解できます。`_` で値を破棄できます。

```
xs = [10, 20, 30]  
  
for i, x in enumerate(xs):
```

```

    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)         # 11, 22

for _, x in enumerate(xs):
    print(x)             # インデックスを破棄

# N要素の分解代入 (3個以上の変数)
triples = [(1, 2, 3), (4, 5, 6)]
for a, b, c in triples:
    print(a + b + c)     # 6, 15

for a, _, c in triples:
    print(a + c)         # 4, 10 (中間の要素を破棄)

```

範囲演算子 (..)

.. 演算子は両端を含む整数の範囲を生成します。1 .. 5 は [1, 2, 3, 4, 5] を生成します。

```

for i in 1 .. 5:
    print(i)             # 1 2 3 4 5

```

例

```

xs = [10, 20, 30]
for x in xs:
    print(x)

s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)             # 0 1 2 3 4

for i in range(2, 6):
    print(i)             # 2 3 4 5

for i in range(0, 10, 2):
    print(i)             # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)             # 10 7 4 1

# マップの走査
m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

# 範囲演算子
for i in 1 .. 3:
    print(i)             # 1 2 3

```

async / await

async fn は並行実行される関数を宣言します。async fn を呼び出すと Task<T> が返ります。別の async fn 内では await を使い、同期コンテキストからは block_on() を使って結果を待ちます。

```
async fn add(a: int, b: int) -> int:
    return a + b

# 同期コンテキストからは block_on() を使用
t: Task<int> = add(20, 22)
print(block_on(t))                # 42
print(block_on(add(1, 2)))        # 3

# async fn 内では await を使用
async fn double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2
```

ルール

- async fn name(...) -> T: の T は await 後の値型です。
 - async fn の呼び出し結果は常に Task<T> です。
 - await expr は Task<T> にのみ使用でき、結果は T です。
 - await は async fn 内でのみ使用可能。同期コンテキストからは block_on(task) を使用します。
 - block_on(task) は現在のスレッドをタスク完了までブロックし、結果を返します。
 - async fn ... -> Unit をサポートします。値を返さない task の待機には block_on(task) を使うのが基本です。
 - task はランタイムの worker pool 上で実行され、task ごとに OS スレッドを作る実装ではありません。
 - async ラムダと async @native fn は v1 では未対応です。
-

@parallel for

@parallel は range(...) または整数 .. を使う counted for ループにだけ付与できます。ループ本体はランタイムのワーカープール上でチャンク単位に並列実行されます。

```
@parallel
for i in range(8):
    print(i)
```

制約

- 対応するのは range(...) と整数 .. のみです。
- 分解代入付きの反復は未対応です。
- 外側スコープのミュータブル変数への代入は拒否されます。
- break と continue は使えません。
- v1 ではループ本体内のインデックス代入とフィールド代入も拒否されます。

ランタイムの worker 数は available_parallelism() で取得できます。

break

- 最も内側のループ (while または for) を即座に脱出する。
- ループの外で使用するとコンパイルエラー。

```
for i in range(10):
    if i == 5:
        break      # i == 5 で脱出
    print(i)       # 0 1 2 3 4
```

エラー例

```
# ループ外での break はコンパイルエラー
break      # Error: break outside loop
```

continue

- 最も内側のループの現在のイテレーションを終了し、次のイテレーションへスキップする。
- ループの外で使用するとコンパイルエラー。

```
for i in range(5):
    if i == 2:
        continue   # i == 2 をスキップ
    print(i)       # 0 1 3 4
```

... (Ellipsis)

- 何もしない文 (no-op)。空ブロックのプレースホルダーとして使用する。
- 関数ボディ、if/elif/else、while、for、match case など任意のブロック内で使用可能。

```
fn not_yet():
    ...

if true:
    ...
else:
    ...
```

match

構文

```
match 式:
    case パターン:
        # 本体
    case パターン if ガード条件:
        # ガード付き本体
    case _:
        # ワイルドカード (何にでもマッチ)
```

パターンの種類

パターン	例	説明
ワイルドカード	<code>_</code>	何にでもマッチ
リテラル	<code>0, "hello", true</code>	値の等値比較

パターン	例	説明
変数束縛	<code>n</code>	何にでもマッチし、変数に束縛
enum バリエント	<code>Color::Red</code>	enum タグの比較（単純な enum）
ADT enum バリエント	<code>Shape::Circle(r)</code>	関連データを持つ enum バリエントにマッチし、束縛する
<code>Some(x)</code>	<code>Some(v)</code>	<code>Option</code> が値ありの場合、中身を束縛
<code>None</code>	<code>None</code>	<code>Option</code> が値なしの場合
<code>Ok(x)</code>	<code>Ok(v)</code>	<code>Result</code> が <code>Ok</code> の場合、中身を束縛
<code>Err(x)</code>	<code>Err(e)</code>	<code>Result</code> が <code>Err</code> の場合、エラー値を束縛
OR パターン	<code>1 \ 2 \ 3</code>	いずれかにマッチ

guard 節

`case` パターン `if` 条件式: の形式でガード条件を指定できる。パターンがマッチし、かつガード条件が真の場合にのみアームが実行される。

OR パターン

複数のパターンを `|` で結合し、いずれかにマッチさせることができます。変数束縛 (`n`、`Some(x)`、`Ok(v)`、`Err(e)`) は OR パターン内では使用できません。

```
match x:
  case 1 | 2 | 3:
    print("small")
  case _:
    print("other")

# enum の OR パターン
match color:
  case Color::Red | Color::Blue:
    print("warm or cool")
  case Color::Green:
    print("green")
```

網羅性チェック

- enum 型: すべてのバリエントをカバーするか `_` が必要。OR パターンの各選択肢は個別にカウントされる。
- Option 型: `Some` と `None` の両方をカバーするか `_` が必要。
- bool 型: `true` と `false` の両方をカバーするか `_` が必要。
- int / float / str リテラル: `_` が必須。
- ガード付きアームは網羅性にカウントされない。

例

```
# enum マッチ
enum Color:
  Red
  Green
```


Blue

```
match color:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")

# Option マッチ
x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")

# Result マッチ
fn divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

match divide(10, 2):
    case Ok(v):
        print(v)          # 5
    case Err(e):
        print(e.message)

# リテラルマッチ
match x:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")

# guard 節
match x:
    case n if n > 0:
        print("positive")
    case n if n < 0:
        print("negative")
    case _:
        print("zero")
```

ADT enum マッチ

enum バリエントが関連データを持つ場合、バインディングパターンを使って値を取り出します。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
```

Point

```
s = Shape::Circle(3.14)
match s:
    case Shape::Circle(r):
        print(r)          # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")
```

複数フィールドを持つバリエーションは、宣言順に各フィールドを別々の名前に束縛します。

スコープルール

- 各 case アームはブロックスコープを持つ。
- 変数束縛パターン (n)、Some(x)、Ok(v)、Err(e) で束縛された変数はそのアーム内でのみ有効。

スコープルール

ブロックスコープ

- if / elif / else / while / for / match の各ブロックはブロックスコープを持つ。
- ブロック内で宣言した変数はブロックの終了と同時にスコープから外れる。

```
for i in range(3):
    tmp = i * 2
# tmp はここではアクセス不可

if true:
    a = 1
# a はここではアクセス不可
```

内側スコープでの再代入

- 内側のスコープで変数に代入すると、外側の変数が変更される (Python スタイルのスコーピング)。
- シャドーイングは行われず、内側の代入は同じ変数を変更する。

```
x = 10
if true:
    x = 99 # 外側の x を変更
    print(x) # 99
print(x) # 99
```

[English](#) | [日本語](#) | [繁體中文](#)

ディレクティブ

ディレクティブは宣言に付与できるコンパイル時メタデータアノテーションです。Java のアノテーションと同様の @name 構文を使用します。

構文

```
@name
@name(key=value, ...)
```

ディレクティブは対象の宣言の前に配置します。複数のディレクティブを重ねることもできます。

対象

ディレクティブは以下の宣言に適用できます:

- `fn` - 関数定義
- `record` - 構造体定義
- 変数宣言 (`@const` 付きまたは通常代入)
- `record` 定義内のフィールド
- `for` - カウント付きループのみ (`@parallel` 用)
- `it` - テストケース定義 (`@each` / `@property` のみ)

組み込みディレクティブ

`@deprecated`

宣言を非推奨としてマークします。非推奨のエンティティが使用（呼び出し、参照、アクセス）されると、コンパイル時に警告が出力されます。

関数に対して:

```
@deprecated
fn old_function() -> int:
    return 42

print(old_function())    # warning: 'old_function' is deprecated
```

型に対して:

```
@deprecated
record OldPoint:
    x: int
    y: int

@const
p = OldPoint(1, 2)    # warning: 'OldPoint' is deprecated
```

変数に対して:

```
@deprecated
@const
old_value = 99

print(old_value)      # warning: 'old_value' is deprecated
```

フィールドに対して:

```
record Config:
    @deprecated
    old_setting: int
    new_setting: int

@const
c = Config(1, 2)
print(c.old_setting)    # warning: 'Config.old_setting' is deprecated
print(c.new_setting)    # 警告なし
```

@const

変数を不変としてマークします。@const で宣言された変数は初期化後に再代入できません。@const なしの場合、変数はデフォルトで可変です。

```
@const
x = 42
# x = 10    # エラー: @const 変数への再代入はできません
```

型アノテーション付き:

```
@const
name: str = "hello"
```

タプル分割代入:

```
@const
a, b = (1, 2)
```

@native

ランタイム（組み込み）によって実装が提供される関数を宣言します。関数本体を持つことはできません。

基本構文:

```
@native
fn contains(string: str, substring: str) -> bool

print(contains("hello world", "world")) # true
```

演算子オーバーロード:

```
@native
fn operator+(a: str, b: str) -> str

print("hello" + " world") # hello world
```

UFCS との組み合わせ:

```
@native
fn to_upper(string: str) -> str

print("hello".to_upper()) # HELLO
```

引数数の検証:

@native 宣言に型シグネチャが含まれている場合、コンパイラは呼び出し時に引数の数を検証します。オーバーロードされた関数（例: 1, 2, 3 引数の range）もサポートされ、いずれかのオーバーロードにマッチすれば検証を通過します。

```
@native
fn range(n: int) -> List<int>
@native
fn range(start: int, end: int) -> List<int>

print(length(range(5)))      # OK: 1 引数のオーバーロードにマッチ
print(length(range(1, 10)))  # OK: 2 引数のオーバーロードにマッチ
print(length(range()))       # Error: expects 1 or 2 argument(s), but got 0
```

標準ライブラリ宣言 (core/):

core/ ディレクトリにはすべての組み込み関数の @native 宣言がカテゴリ別に格納されています:

ファイル	内容
core/builtins.ry	print, length, range, enumerate, zip, exit, args, available_parallelism, sleep
core/str.ry	contains, starts_with, ends_with, find, substring, char_at, replace, to_upper, to_lower, trim, trim_start, trim_end, repeat, reverse, split, join
core/convert.ry	to_int, to_float, to_str
core/list.ry	append, pop, insert, remove_at, slice, distinct, flatten, sort, first, last, is_empty
core/map.ry	keys, values, items, has_key, get, merge
core/set.ry	add, remove, union, intersection, difference, symmetric_difference, is_subset, is_superset
core/higher_order.ry	filter, map, reduce, fold, any, all, sum, min, max

これらのファイルは ry 実行バイナリの近くに core/ ディレクトリが存在する場合、プレリユードとして自動的にロードされます。プレリユードにより、組み込み関数呼び出し時の引数数検証が有効になります。

制約事項: - @native 関数は本体を持ってません (シグネチャの後に : を付けるとエラー)。- 本体を付けるとパースエラー: @native function must not have a body。- 宣言した関数は既存の組み込み関数に対応している必要があります。対応していない場合はコンパイル時にエラーになります。

将来の拡張方向: - @native("libfoo.so") —外部共有ライブラリへの FFI バインディング。

@parallel

カウント付き for ループを並列実行対象としてマークします。

```
@parallel
for i in range(8):
    work(i)
```

対応対象:

- for 文のみ

制約事項:

- for 文には 1 つの @parallel だけ指定できます。
- 反復対象は range(...) または整数 .. レンジに限られます。
- 分解代入付きの反復は未対応です。
- 外側の可変変数への代入は拒否されます。
- v1 ではループ本体内の break、continue、インデックス代入、フィールド代入は拒否されます。

@each

パラメタライズドテストを有効にし、it ブロックを異なるパラメータで複数回実行します。

構文:

```
@each([(arg1, arg2, ...), ...])
it("description with {0} and {1}", fn(param1: type, param2: type):
    # テスト本体
)
```

対応対象: it 呼び出しのみ

制約事項: - 引数はタプルのリストである必要がある - タプルのアリティはラムダのパラメータ数と一致する必要がある - 説明文の {0}, {1}, ... はパラメータ値の文字列表現で置換される

@property

プロパティベーステストを有効にし、it ブロックにランダム入力を生成します。

構文:

```
@property(count=100)
it("property name", fn(a: int, b: int):
    # ランダム値によるテスト本体
)
```

対応対象: it 呼び出しのみ

パラメータ:

パラメータ	型	デフォルト	説明
count	int	100	ランダム試行回数

対応するパラメータ型:

型	範囲
int	-1000 ~ 1000
float	-1000.0 ~ 1000.0
bool	true または false
str	ランダム ASCII、0-20 文字

失敗時は反例（失敗を引き起こしたパラメータ値）が表示されます。

@inline

LLVM オプティマイザにインライン化のヒントを与えます。デフォルトでは、関数を積極的にインライン化するように指示します。

基本的な使い方（常にインライン化）:

```
@inline
fn add(a: int, b: int) -> int:
    return a + b
```

mode パラメータ付き:

```
@inline(mode="always")
fn hot_path(x: int) -> int:
    return x * 2 + 1
```

```
@inline(mode="hint")
fn medium_path(x: int) -> int:
    return x + 1
```

```
@inline(mode="never")
fn cold_error_handler(msg: str):
    print("ERROR: " + msg)
```

モード:

モード	LLVM 属性	説明
always (デフォルト)	AlwaysInline	常にインライン化する
hint	InlineHint	オプティマイザにインライン化を提案する
never	NoInline	インライン化を禁止する

制約: - @inline は @native と併用できません (native 関数にはインライン化するボディがありません)。- 不明な mode 値はコンパイルエラーになります。

パラメータ (将来拡張)

ディレクティブは将来の拡張に備え、パラメータ構文をサポートしています:

```
@deprecated(reason="use new_api instead")
fn old_api() -> int:
    return 0
```

現時点では、パラメータはパースされますが @deprecated ディレクティブでは使用されません。

注意事項

- 非推奨のエンティティは正常に動作します。警告が出力されるだけです。
- 警告は使用箇所では出力され、定義箇所では出力されません。
- 非推奨のエンティティを定義しても、使用しなければ警告は出力されません。
- 未知のディレクティブ名はパースエラーになります。
- サポートされない対象 (if、while 等) にディレクティブを付与するとパースエラーになります。for に使えるのは @parallel のみです。

[English](#) | [日本語](#) | [繁體中文](#)

エラーリファレンス

エラーフォーマット

ry はコンパイルエラーを Rust 風のリッチフォーマットで表示します。エラーの正確な位置がソースコードのコンテキストとともに表示されます:

```
error: cannot reassign @const variable: x
--> main.ry:5:1
|
5 | x = 10
|   ^ cannot reassign @const variable: x
```

各エラーメッセージには以下が含まれます: - エラーレベルとメッセージ (error: ...) - ファイル位置 (--> ファイル: 行: 列) - ソース行 (該当するコード) - キャレットインジケーター (^) 正確な列を指し示す

コンパイルエラー一覧

エラー内容	原因	例
未宣言変数の使用	宣言されていない変数を参照した	print(x) (x が未宣言)
@const 変数への再代入	@const で宣言した変数に再代入した	@const x = 1 → x = 2
同名変数の再宣言	同スコープで同名の変数を再宣言した	x = 1 → 同名の再宣言

エラー内容	原因	例
変数の型変更再代入	変数に異なる型の値を代入した	<code>x = 1 → x = 3.14</code>
型アノテーション不一致	宣言時の型と代入値の型が異なる	<code>x: int = 3.14</code>
オーバーロード戻り値型の重複	引数型が同一で戻り値型のみ異なるオーバーロードを定義した	引数 (<code>int</code> , <code>int</code>) で戻り値が <code>int</code> と <code>float</code> の2つの関数
オーバーロード解決失敗	引数型に一致するオーバーロードが存在しない	<code>fn add(a: int, b: int)</code> に <code>add(1.0, 2.0)</code>
ビット演算に <code>float</code> を渡す	<code>&</code> , <code> </code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code> に <code>float</code> 型を渡した	<code>3.14 & 1</code>
空リスト	空リストリテラル <code>[]</code> から型を推論できない	<code>xs = []</code>
空マップ	空マップリテラル <code>{}</code> から型を推論できない	<code>m = {}</code>
タプル範囲外インデックス	タプルの存在しないインデックスにアクセスした	<code>t = (1, 2) → t.2</code>
ループ外での <code>break/continue</code>	<code>for/while</code> ループの外で <code>break</code> または <code>continue</code> を使用した	関数トップレベルで <code>break</code>
ブロック内でのモジュールインポート	<code>from</code> 文を関数・条件ブロック内で使用した	関数内で <code>from math</code>
循環インポート	モジュールが相互にインポートし合っている	<code>a.ry</code> が <code>b.ry</code> を、 <code>b.ry</code> が <code>a.ry</code> をインポート
同一フィールド名の重複	構造体内で同じフィールド名を2回定義した	<code>type T: x: int</code> の中に <code>x</code> を2つ定義
<code>match</code> の網羅性不足	<code>match</code> がすべてのパターンを網羅していない	<code>enum</code> の一部バリエーションが未カバー、 <code>Option</code> で <code>None</code> が欠落、リテラルに <code>_</code> なし
? の非 <code>Result</code> 型への適用	<code>Result</code> 型でない式に? を適用した	<code>x = 42 → x?</code>
? の非 <code>Result</code> 返却関数での使用	<code>Result</code> を返さない関数内で? を使用した	<code>fn foo() -> int: → bar()?</code>

コンパイルエラーの例

```
# @const 変数への再代入
@const
x = 10
x = 20    # エラー

# 型変更再代入
n = 1
n = "hello"    # エラー: int 型に str 型を代入

# 空リスト
xs = []    # エラー: 型推論不可

# ループ外での break
break    # エラー: ループの外
```



```
# ブロック内でのインポート
fn foo():
    from math    # エラー: トップレベルのみ

# 同一フィールド名の重複
record Bad:
    x: int
    x: float    # エラー: xが重複
```

ランタイムエラー一覧

エラー内容	原因	例
リスト範囲外アクセス	リストのインデックスが範囲を超えている	<code>xs = [1, 2, 3] → xs[5]</code>
マップ存在しないキーアクセス	マップに存在しないキーを参照した	<code>m = {"a": 1} → m["b"]</code>
契約違反	<code>require</code> ・ <code>ensure</code> ・ <code>invariant</code> の条件が <code>false</code> と評価された	契約による設計 を参照

すべてのランタイムエラーはプロセスを `exit(1)` で終了します。

ランタイムエラーの例

```
# リスト範囲外アクセス
xs = [1, 2, 3]
print(xs[10])    # ランタイムエラー: exit(1)

# マップ存在しないキーアクセス
m = {"a": 1}
print(m["z"])    # ランタイムエラー: exit(1)
```

[English](#) | [日本語](#) | [繁體中文](#)

関数リファレンス

関数定義の構文

```
fn 関数名 (引数名: 型, ...) -> 戻り値型:
    # 本体
    return 値
```

- 引数の型は省略可能。省略時は `any` 型として扱われる。
- 戻り値型は省略可能。省略時はボディから推論される（名前付き関数とラムダの両方）。`return` 文がなければ `Unit` に推論される。明示的に任意の戻り値型を許容するには `-> any` を指定する。
- 本体はインデントされたブロック。
- 明示的な戻り値型（`Unit` と `any` を除く）を持つ関数は、すべての制御フローパスで `return` 文が必要です。不足している場合はコンパイルエラーになります。
- 関数には `require`（事前条件）と `ensure`（事後条件）を定義できます。[契約による設計](#)を参照。

命名規則: 関数名と引数名は `snake_case`（例: `add`、`get_value`、`map_list`）を使用する必要があります。コンパイラがこの規則を強制します。

```
fn add(a: int, b: int) -> int:
    return a + b

fn greet(name: str) -> Unit:
    print("Hello, " + name)    # 戻り値型は Unit (明示的)
```

引数と戻り値の型

項目	説明
引数型	省略可能。: 型 を省略すると any になる
戻り値型	省略可能。省略時はボディから推論される (return 文がなければ Unit)
Unit	値を返さない関数の戻り値型

注意: 関数の引数は不変です。関数ボディ内で引数を再代入することはできません。これにより、事後条件チェックで引数のエントリ時の値を常に利用できます ([契約による設計](#)を参照)。

```
fn no_return(x: int) -> Unit:    # 戻り値型 Unit (明示的)
    print(x)

fn get_value() -> int:          # 戻り値型 int
    return 42

fn identity(x) -> any:          # 引数型 any (省略)
    return x
```

型省略と any

引数の型アノテーションを省略すると、その引数は any として扱われます。any は実行時に任意のプリミティブ値を受け入れる動的型です。Python の型なし引数と同様の仕組みです。

```
# すべての引数が any
fn add(a, b):
    return a + b

add(1, 2)                # 3 (int + int)
add("hello", " world")   # "hello world" (str + str)
add(1, 2.0)               # 3.0 (int + float)
```

型アノテーションに any を明示的に書くこともできます:

```
fn identity(x: any) -> any:
    return x
```

戻り値型の推論

戻り値型を省略すると、ボディ内の return 文から推論されます:

```
fn double(x: int):          # 戻り値型は int に推論
    return x * 2

fn greet(name: str):        # 戻り値型は Unit に推論 (return なし)
    print("Hello, " + name)
```

明示的に任意の戻り値型を許容するには -> any を指定します:

```
fn flexible(x: any) -> any:
    return x    # int、float、str 等を返せる
```

再帰

関数は自身を呼び出せる。

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

末尾呼び出し最適化

コンパイラは、関数の最後の処理が自分自身の呼び出しである自己再帰末尾呼び出しを自動的に検出し、LLVM の `musttail` 最適化を適用します。これにより、末尾再帰関数は一定のスタック空間を使用することが保証され、深い再帰でのスタックオーバーフローを防ぎます。

```
fn sum_to(n: int, acc: int) -> int:
    if n <= 0:
        return acc
    return sum_to(n - 1, acc + n)    # 末尾呼び出し → 最適化される
```

```
sum_to(1000000, 0)    # スタックオーバーフローなしで動作
```

TCO の条件:

- `return` 文で自分自身を直接呼び出している (`return f(args)`)
- 呼び出しの結果がそのまま返される (`return n * f(n-1)` は末尾呼び出しではない)
- 関数に `ensure` (事後条件) がない

相互再帰 (A が B を呼び、B が A を呼ぶ) は現在最適化されません。

オーバーロード

引数の数や型が異なる同名の関数を複数定義できる。

ルール

- 引数の数または型が異なれば同名の関数を定義可能。
- 呼び出し時は引数の型と数に基づいて適切な関数が選択される。
- 戻り値型だけが異なるオーバーロードはできない。

```
fn area(side: int) -> int:
    return side * side
```

```
fn area(w: int, h: int) -> int:
    return w * h
```

```
a = area(5)        # 25
b = area(3, 4)     # 12
```

解決優先順位

複数のオーバーロードが呼び出しに一致する場合、コンパイラは以下の優先順位（高い順）で最も具体的なものを選択します：

1. 完全型一致 – 引数の型がパラメータの型に完全に一致
2. 暗黙の拡大変換 – 安全な拡大変換 ($u8 \rightarrow int$ 、 $u8 \rightarrow float$ 、 $int \rightarrow float$)
3. **union 型一致** – 引数の型が union パラメータ型のメンバー
4. **any 型一致** – パラメータの型が any（任意の値を受け入れる）

完全一致が最も多いオーバーロードが選ばれます。2 つ以上のオーバーロードが同じ具体性を持つ場合、コンパイラは曖昧性エラーを報告します。

低レベル数値型 ($i8$ 、 $i16$ 、 $i32$ 、 $i64$ 、 $u8 \sim u64$ 、 $f32$) は暗黙の拡大変換に**参加しません**– 明示的な `as` キャストが必要です。

```
fn process(x: int) -> str:
    return "int"

fn process(x) -> str:           # x: any
    return "any"

process(42)           # "int" – 完全一致 (int) が any に勝つ
process("hello")      # "any" – str の完全一致がないため any にフォールバック

fn double(x: float) -> float:
    return x * 2.0

double(5)             # OK – int が暗黙的に float に拡大変換され、10.0 を返す
```

デフォルト引数

パラメータにデフォルト値を指定でき、呼び出し時に末尾の引数を省略できる。

構文

```
fn connect(host: str, port: int = 8080, timeout: int = 30):
    # ...

connect("localhost")           # port=8080, timeout=30
connect("localhost", 3000)     # port=3000, timeout=30
connect("localhost", 3000, 10000) # port=3000, timeout=10000
```

ルール

- デフォルトパラメータは非デフォルトパラメータの後に配置する。
- デフォルト値を持つパラメータには明示的な型注釈が**必須**（例: `x: int = 10`; `x = 10` はコンパイルエラー）。
- デフォルト値はコンパイル時定数式（リテラルおよび `@const` 変数）に限定。
- デフォルト引数により曖昧なオーバーロード（arity 範囲の重複 + 型の一致）が生じる場合、コンパイルエラーとなる。

エラー：曖昧なオーバーロード

```
fn calc(x: int, y: int = 0) -> int:
    return x + y
fn calc(x: int) -> int:           # 上の calc(int) と衝突
    return x * 2
```

制限事項

- ジェネリック関数およびラムダ式ではデフォルト引数は未サポート。
-

Unit 型関数

戻り値のない関数は Unit を返す。戻り値型は省略可能 (Unit に推論される)、または `-> Unit` で明示的に指定可能。

```
fn log(msg: str) -> Unit:
    print(msg)
```

Task と async 関数

`Task<T>` は並行実行の組み込みハンドル型です。 `async fn` は `Task<T>` を返し、`await` は別の `async fn` 内で `T` を取り出します。 `block_on(task)` は同期コンテキストからタスクの完了までブロックします。

```
async fn add(a: int, b: int) -> int:
    return a + b
```

同期コンテキストからは `block_on()` を使用

```
t: Task<int> = add(20, 22)
```

```
print(block_on(t))
```

42

```
block_on(add(1, 2))
```

待機して結果を捨てる

`async fn` 内では `await` を使用

```
async fn double_add(a: int, b: int) -> int:
```

```
    return (await add(a, b)) * 2
```

ルール

- `async fn name(...) -> T:` の `T` は `await` 後の値型です。
 - `async fn` の呼び出し結果は常に `Task<T>` です。
 - `await expr` は `Task<T>` にのみ使用でき、結果は `T` です。
 - `await` は `async fn` 内でのみ使用可能。同期コンテキストからは `block_on(task)` を使用します。
 - `block_on(task)` は現在のスレッドをタスク完了までブロックし、結果を返します。
 - `async fn ... -> Unit` をサポートします。値を返さない `task` の待機には `block_on(task)` を使うのが基本です。
 - `task` はランタイムの worker pool 上で実行され、`task` ごとに OS スレッドを作る実装ではありません。
 - `async` ラムダと `async @native fn` は v1 では未対応です。
-

ラムダ関数

無名関数をその場で定義できる。

構文

単一式 (式の値が返る。戻り値型は推論)

```
fn(引数名: 型, ...) => 式
```

引数型の省略 (*any* がデフォルト)

```
fn(引数名, ...) => 式
```

```
# 複数行ブロック
fn(引数名: 型, ...):
    # 複数の文
    return 値

# 戻り値型の明示（省略可能）
fn(引数名: 型, ...) -> 戻り値型 => 式
```

例

```
double = fn(x: int) => x * 2
result = double(5)    # 10

add = fn(a: int, b: int) => a + b
sum = add(3, 4)       # 7

# 複数行ラムダ
abs = fn(x: int):
    if x < 0:
        return -x
    return x
```

クロージャ

ラムダ関数は定義された時点の外側スコープの変数を**値でキャプチャ**する。

```
base = 10
add_base = fn(x: int) => x + base    # base を値でキャプチャ

base = 99                            # キャプチャ済みの値には影響しない
r = add_base(5)                      # 15 (キャプチャ時の base = 10 を使用)
```

キャプチャルール

項目	内容
キャプチャ方式	値キャプチャ（コピー）
キャプチャタイミング	ラムダ定義時
外側変数の変更の影響	ない（コピーのため）

関数型

関数を値として扱うための型。

構文

```
fn(引数型 1, 引数型 2, ...) -> 戻り値型
```

例

```
f: fn(int) -> int = fn(x: int) => x * 2
g: fn(int, int) -> int = fn(a: int, b: int) => a + b

fn apply(func: fn(int) -> int, x: int) -> int:
    return func(x)

result = apply(f, 5)    # 10
```

高階関数

関数を引数として受け取ったり、戻り値として返したりできる。

```
fn map_list(xs: List<int>, f: fn(int) -> int) -> List<int>:
    result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

doubled = map_list([1, 2, 3], fn(x: int) => x * 2)
# [2, 4, 6]
```

ジェネリック関数

関数は型パラメータを持つことができ、コード重複なしに型安全な再利用を実現します。

構文

```
fn name<T, U>(param1: T, param2: U) -> T:
    # T, U を型として使用するボディ
```

例

```
fn identity<T>(x: T) -> T:
    return x

# 明示的な型引数
result = identity[int](42)        # 42
result = identity[str]("hello")   # "hello"

# 型推論 (実引数から型引数を推論)
result = identity(42)             # T = int, result = 42
result = identity("hello")        # T = str, result = "hello"
```

複数の型パラメータ

```
fn pick_first<T, U>(a: T, b: U) -> T:
    return a

result = pick_first(1, "x")        # T = int, U = str, result = 1
result = pick_first("hello", 42)   # T = str, U = int, result = "hello"
```

型制約（バウンド）

型パラメータには `RecordName` 構文でレコード型の制約を付けることができます。具体型はバウンド型自体またはそのサブタイプでなければなりません。

```
record Animal:
  name: str
  legs: int

record Dog < Animal:
  breed: str

fn get_name<T: Animal>(a: T) -> str:
  return a.name

get_name(Dog("Rex", 4, "Lab")) # OK — Dog は Animal のサブタイプ
get_name(Animal("Cat", 4))    # OK — 完全な型一致
```

バウンド付きとバウンドなしの型パラメータを混在させることができます:

```
fn pair_name<T: Animal, U>(a: T, x: U) -> str:
  return a.name
```

動作の仕組み

ジェネリック関数は**単相化**を使用します: 型引数の一意な組み合わせごとに特殊化されたバージョンの関数が生成されます。同じインスタンス化はキャッシュされ、複数の呼び出しで再利用されます。型制約がある場合、インスタンス化時に検証されます。

UFCS（統一関数呼び出し構文）

`a.f(b)` の形式で `f(a, b)` を呼び出せる。メソッドチェーンに使いやすい。

構文

```
# 通常呼び出し
f(a, b)
```

```
# UFCS 呼び出し（等価）
a.f(b)
```

チェーン

```
fn double(x: int) -> int:
  return x * 2

fn add_one(x: int) -> int:
  return x + 1

result = 5.double().add_one() # double(5) → 10, add_one(10) → 11
```

フィールドアクセスとの混在

フィールドアクセス (`.field`) と UFCS (`.method()`) は同じドット記法で書けるが、引数の有無で区別される。

```
p = Point(3, 4)
length = p.x.to_float() # フィールドアクセス + UFCS
```


演算子オーバーロード

ユーザー定義型に対して演算子の振る舞いを定義できる。

構文

```
# 二項演算子 (引数 2 個)
fn operator<op>(a: 型, b: 型) -> 戻り値型:
    ...

# 単項演算子 (引数 1 個)
fn operator<op>(a: 型) -> 戻り値型:
    ...
```

オーバーロード可能な演算子

種別	演算子
算術 (二項)	+ - * / % ** //
比較 (二項)	== != < <= > >=
ビット (二項)	& \ ^ << >>
論理 (二項)	and or
所属	in
添字	[] (読み取り)、[] = (書き込み)
呼び出し	()
キャスト	as
単項	- ~ not

戻り値型の制約

比較演算子、論理演算子、所属演算子は bool を返す必要がある:

種別	演算子	必須戻り値型
比較	== != < <= > >=	bool
論理	and or not	bool
所属	in	bool
キャスト	as	必須 (ターゲット型)

```
# OK
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

# エラー: comparison operator '==' must return 'bool', but returns 'int'
fn operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算術演算子・ビット演算子には戻り値型の制約はない。

二項 / 単項の区別

引数の個数で区別する。

```

record Vec2:
  x: float
  y: float

# 二項 +
fn operator+(a: Vec2, b: Vec2) -> Vec2:
  return Vec2(a.x + b.x, a.y + b.y)

# 単項 -
fn operator-(v: Vec2) -> Vec2:
  return Vec2(-v.x, -v.y)

# 比較
fn operator==(a: Vec2, b: Vec2) -> bool:
  return a.x == b.x and a.y == b.y

v1 = Vec2(1.0, 2.0)
v2 = Vec2(3.0, 4.0)
v3 = v1 + v2      # Vec2(4.0, 6.0)
v4 = -v1          # Vec2(-1.0, -2.0)

```

チェック付き/飽和演算

低レベル整数型 (i8, i16, i32, i64, u8, u16, u32, u64) 向けの明示的なオーバーフロー制御用組み込み関数です。両方の引数は同じ型でなければなりません。

関数	戻り値	動作
checked_add(a, b)	Result<T, Error>	オーバーフロー時に Err を返す
checked_sub(a, b)	Result<T, Error>	アンダーフロー時に Err を返す
checked_mul(a, b)	Result<T, Error>	オーバーフロー時に Err を返す
saturating_add(a, b)	T	オーバーフロー時に型の最小/最大値にクランプ
saturating_sub(a, b)	T	アンダーフロー時に型の最小/最大値にクランプ
saturating_mul(a, b)	T	オーバーフロー時に型の最小/最大値にクランプ
wrapping_add(a, b)	T	明示的なラッピング (+ と同じ)
wrapping_sub(a, b)	T	明示的なラッピング (- と同じ)
wrapping_mul(a, b)	T	明示的なラッピング (* と同じ)

```

# チェック付き: Result を返す。match や ? でハンドリング
r = checked_add(2147483647i32, 1i32)
match r:
  case Ok(v):
    print(v)
  case Err(e):
    print("overflow!")    # "overflow!" が出力される

# 飽和: 境界値にクランプ
v = saturating_add(2147483647i32, 100i32)
print(v as int)    # 2147483647

```

```
# ラッピング: 自己文書化的なラッピング動作
v = wrapping_add(2147483647i32, 1i32)
print(v as int)    # -2147483648
```

注意: これらの関数は浮動小数点型 (f32) や高レベルの int 型をサポートしていません。低レベル整数のデフォルトの +、-、* 演算子はラッピング動作 (符号付きは 2 の補数、符号なしはモジュラー) を使用します。

[English](#)

HTTP リファレンス

型

型	説明
HttpRequest	受信 HTTP リクエストの不透明ハンドル
HttpResponse	送信 HTTP レスポンスの不透明ハンドル
HttpClientResponse	HTTP クライアントレスポンスの不透明ハンドル

HttpRequest はサーバーフレームワークから提供されます。HttpResponse は `http_response()` で作成します。HttpClientResponse はクライアント関数 (`http_get`、`http_post`、`http_request`) から返されます。

関数 (http パッケージ)

これらの関数は明示的なインポートが必要です:

```
from http import http_listen, http_method, http_path, http_header, http_body, http_query, http_query_all
```

サーバー

関数	シグネチャ	説明
<code>http_listen</code>	<code>(host: str, port: int, handler: fn(HttpRequest) -> HttpResponse) -> Unit</code>	指定アドレスで HTTP サーバーを起動します。accept ループでブロックし、リクエストごとに handler を呼び出します。
<code>http_listen</code>	<code>(host: str, port: int, handler: fn(HttpRequest) -> HttpResponse, max_requests: int) -> Unit</code>	<code>max_requests</code> 件のリクエストを処理した後に停止する HTTP サーバーを起動します。 <code>async fn + block_on()</code> によるライフサイクル管理が可能になります。
<code>http_listen</code>	<code>(host: str, port: int, handler: fn(HttpRequest) -> HttpResponse, max_requests: int, port_callback: fn(int) -> Unit) -> Unit</code>	上記と同じですが、 <code>bind + listen</code> 成功後に実際にバインドされたポートで <code>port_callback</code> を呼び出します。OS が割り当てるエフェメラルポートを使用するにはポート 0 を指定します。

リクエストアクセサ

関数	シグネチャ	説明
<code>http_method</code>	<code>(req: HttpRequest) -> str</code>	HTTP メソッドを返します (例: "GET"、"POST")。

関数	シグネチャ	説明
<code>http_path</code>	<code>(req: HttpRequest) -> str</code>	クエリ文字列を除いたリクエストパスを返します (例: <code>"/search?q=hello"</code> の場合 <code>"/search"</code>)。
<code>http_header</code>	<code>(req: HttpRequest, key: str) -> Option<str></code>	リクエストヘッダーの値を返します (大文字小文字を区別しない検索)。見つからない場合は <code>None</code> を返します。
<code>http_body</code>	<code>(req: HttpRequest) -> str</code>	リクエストボディを文字列として返します。
<code>http_query</code>	<code>(req: HttpRequest, key: str) -> Option<str></code>	クエリパラメータの値を返します。見つからない場合は <code>None</code> を返します。値は自動的に URL デコードされます。
<code>http_query_all</code>	<code>(req: HttpRequest) -> Map<str, str></code>	すべてのクエリパラメータをマップとして返します。キーと値は自動的に URL デコードされます。
<code>http_cookie</code>	<code>(req: HttpRequest, name: str) -> Option<str></code>	名前で指定した Cookie の値を返します。見つからない場合は <code>None</code> を返します。
<code>http_cookies</code>	<code>(req: HttpRequest) -> Map<str, str></code>	すべての Cookie をマップとして返します。
<code>http_form_field</code>	<code>(req: HttpRequest, name: str) -> Option<str></code>	マルチパートフォームのテキストフィールドの値を返します。見つからない場合は <code>None</code> を返します。
<code>http_form_file</code>	<code>(req: HttpRequest, name: str) -> Option<Map<str, str>></code>	ファイルアップロード情報を <code>Option</code> として返します。 <code>Some(map)</code> には <code>"filename"</code> 、 <code>"content_type"</code> 、 <code>"data"</code> のキーが含まれます。見つからない場合は <code>None</code> を返します。
<code>http_form_fields</code>	<code>(req: HttpRequest) -> Map<str, str></code>	すべてのマルチパートフォームのテキストフィールドをマップとして返します。

レスポンスビルダー

関数	シグネチャ	説明
<code>http_response</code>	<code>(status: int, headers: Map<str, str>, body: str) -> HttpResponse</code>	指定したステータスコード、ヘッダー、ボディで HTTP レスポンスを作成します。

使用例

基本的な HTTP サーバー

```
from http import http_listen, http_method, http_path, http_header, http_body, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    method = http_method(req)
    path = http_path(req)
    if path == "/hello":
        return http_response(200, {"Content-Type": "text/plain"}, "Hello, World!")
    if path == "/echo":
        body = http_body(req)
        return http_response(200, {"Content-Type": "text/plain"}, body)
```

```
    return http_response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

async fn によるノンブロッキングサーバー

```
from http import http_listen, http_path, http_response
```

```
async fn start_server(port: int) -> str:
    http_listen("127.0.0.1", port, fn(req: HttpRequest) -> HttpResponse:
        path = http_path(req)
        if path == "/api/health":
            return http_response(200, {"Content-Type": "application/json"}, {"\status\": \ok\"})
            return http_response(404, {"Content-Type": "text/plain"}, "Not Found")
    )
    return "done"
```

```
t = start_server(8080)
```

サーバーはバックグラウンドタスクとして実行される

リクエスト制限付きサーバー (max_requests)

```
from http import http_listen, http_path, http_response, http_get, http_client_status, http_client_body
```

```
port_holder = [0]
```

```
fn on_port(p: int) -> Unit:
    port_holder[0] = p
```

```
async fn start_server() -> str:
    http_listen("127.0.0.1", 0, fn(req: HttpRequest) -> HttpResponse:
        return http_response(200, {"Content-Type": "text/plain"}, "Hello!")
    , 1, on_port) # 1 リクエスト後に停止; on_port にバインドされたポートを通知
    return "done"
```

```
t = start_server()
```

```
sleep(100) # サーバーの起動を待機
```

```
port = port_holder[0]
```

```
match http_get("http://127.0.0.1:" + to_str(port) + "/"):
    case Ok(resp):
        print(http_client_body(resp)) # "Hello!"
    case Err(e):
        print("error")
```

```
result = block_on(t) # サーバーは 1 リクエスト後に終了; block_on が完了する
```

クエリパラメータの読み取り

```
from http import http_listen, http_path, http_query, http_query_all, http_response
```

```
http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    path = http_path(req)
    if path == "/search":
        match http_query(req, "q"):
            case Some(query):
                return http_response(200, {"Content-Type": "text/plain"}, "Search: " + query)
```

```

        case None:
            return http_response(400, {"Content-Type": "text/plain"}, "Missing query parameter: q")
    return http_response(404, {"Content-Type": "text/plain"}, "Not Found")
)

```

ヘッダーの読み取り

```

from http import http_listen, http_header, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    match http_header(req, "Authorization"):
        case Some(token):
            return http_response(200, {"Content-Type": "text/plain"}, "Authenticated: " + token)
        case None:
            return http_response(401, {"Content-Type": "text/plain"}, "Unauthorized")
)

```

フォーム送信の処理

```

from http import http_listen, http_form_field, http_form_file, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    match http_form_field(req, "username"):
        case Some(name):
            match http_form_file(req, "avatar"):
                case Some(file_info):
                    filename = file_info["filename"]
                    return http_response(200, {"Content-Type": "text/plain"}, "Hello " + name + ", file: " + filename)
                case None:
                    return http_response(400, {"Content-Type": "text/plain"}, "No file uploaded")
        case None:
            return http_response(400, {"Content-Type": "text/plain"}, "Missing username")
)

```

Cookie の読み取り

```

from http import http_listen, http_cookie, http_cookies, http_response

http_listen("127.0.0.1", 8080, fn(req: HttpRequest) -> HttpResponse:
    match http_cookie(req, "session_id"):
        case Some(sid):
            return http_response(200, {"Content-Type": "text/plain"}, "Session: " + sid)
        case None:
            return http_response(401, {"Content-Type": "text/plain"}, "No session")
)

```

動作仕様

- `http_listen()` はアドレスにバインドし、リスニングを開始して accept ループに入ります。
- 引数 3 つで呼び出した場合、accept ループは無期限に実行されます。
- 引数 4 つで呼び出した場合 (`max_requests`)、指定した数のリクエストを処理した後にサーバーが停止します。`max_requests` は正の整数でなければなりません。これにより `async fn + block_on()` によるライフサイクル管理が可能になります。不正なリクエスト (サイレントにスキップされる) は制限のカウントに含まれません。

- 引数 5 つで呼び出した場合 (`max_requests`、`port_callback`)、`bind + listen` 成功後に `port_callback` が実際にバインドされたポートで同期的に呼び出されます。並列テストでのポート競合を避けるため、ポート 0 (OS が割り当てるエフェメラルポート) と組み合わせて使用できます。
- サーバーはデフォルトで HTTP/1.1 キープアライブをサポートします。単一の接続で複数のリクエストを処理できます。サーバーは各リクエストの `Connection` ヘッダーを確認します: `Connection: close` が送信された場合、レスポンス後に接続が閉じられます。それ以外の場合、接続は後続のリクエストのために維持されます。アイドル接続は 5 秒のタイムアウト後に閉じられます。
- `Content-Length` がヘッダーマップに指定されていない場合、レスポンスに自動的に追加されます。
- サーバーは `Content-Length` ベースのボディ読み取りと `Transfer-Encoding: chunked` デコードを含む HTTP/1.1 をサポートします。
- リクエストに `Transfer-Encoding: chunked` が存在する場合、ボディは自動的にデコード・結合されます。Ry コードは完全なボディを透過的に受け取ります。
- `Transfer-Encoding: chunked` と `Content-Length` の両方が存在する場合、リクエストは不正として拒否されます (RFC 9112 準拠)。
- チャンクレスポンスを送信するには、`http_response()` に渡すヘッダーマップに `"Transfer-Encoding": "chunked"` を含めます。ボディは自動的にチャンク形式でエンコードされます。
- `http_header()` によるヘッダー検索は大文字小文字を区別しません。
- `http_path()` はクエリ文字列を除いたパスを返します。クエリパラメータは `http_query()` または `http_query_all()` で別途アクセスします。
- クエリパラメータの値は自動的に URL デコードされます (%20 →スペース、+ → スペース)。
- 重複するクエリパラメータキーの場合、最初の値が返されます。
- `http_cookie()` と `http_cookies()` は `Cookie` ヘッダーを ; で分割し、各ペアを最初の = で分割してパースします。名前と値の先頭と末尾の空白はトリムされます。
- 重複する `Cookie` 名の場合、最初の値が返されます。
- `Cookie` の値には = 文字を含めることができます (最初の= のみが名前と値の区切りとなります)。
- `http_form_field()`、`http_form_file()`、`http_form_fields()` は `multipart/form-data` リクエストボディをパースします。パースは遅延実行され、最初の呼び出し時にパースされてキャッシュされます。
- `boundary` パラメータは `Content-Type` ヘッダーから抽出され、クォート付きとクォートなしの両方の値をサポートします。
- `Content-Disposition` に `filename` を持つパートはファイルアップロードとして扱われ、持たないパートはテキストフィールドとして扱われます。
- 重複するフィールド/ファイル名の場合、最初の値が返されます。
- `http_form_file()` は `"filename"`、`"content_type"`、`"data"` のキーを持つ `Some(map)` を返すか、フィールドが見つからない場合は `None` を返します。パートに `Content-Type` が指定されていない場合、デフォルトで `"application/octet-stream"` になります。
- マルチパートでないリクエストの場合、フォーム関数は `None` (`http_form_field`、`http_form_file`) または空のマップ (`http_form_fields`) を返します。

サポートされるステータスコード

以下のステータスコードには標準の理由フレーズがあります (RFC 9110) :

コード	理由
100	Continue
101	Switching Protocols
200	OK
201	Created
202	Accepted
203	Non-Authoritative Information
204	No Content
205	Reset Content
206	Partial Content
300	Multiple Choices
301	Moved Permanently

コード	理由
302	Found
303	See Other
304	Not Modified
307	Temporary Redirect
308	Permanent Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
408	Request Timeout
409	Conflict
410	Gone
411	Length Required
413	Content Too Large
414	URI Too Long
415	Unsupported Media Type
416	Range Not Satisfiable
417	Expectation Failed
422	Unprocessable Content
426	Upgrade Required
429	Too Many Requests
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported

その他のステータスコードは理由フレーズとして "Unknown" を使用します。

HTTP クライアント

クライアント関数

関数	シグネチャ	説明
<code>http_get</code>	<code>(url: str) -> Result<HttpClientResponse, Error></code>	指定した URL に HTTP GET リクエストを送信します。
<code>http_post</code>	<code>(url: str, body: str, headers: Map<str, str>) -> Result<HttpClientResponse, Error></code>	ボディとヘッダーを指定して HTTP POST リクエストを送信します。
<code>http_request</code>	<code>(method: str, url: str, headers: Map<str, str>, body: str) -> Result<HttpClientResponse, Error></code>	カスタムメソッドで HTTP リクエストを送信します。

レスポンスアクセサ

関数	シグネチャ	説明
<code>http_client_status</code>	<code>(resp: HttpClientResponse) -> int</code>	HTTP ステータスコードを返します。
<code>http_client_body</code>	<code>(resp: HttpClientResponse) -> str</code>	レスポンスボディを文字列として返します。
<code>http_client_header</code>	<code>(resp: HttpClientResponse, key: str) -> Option<str></code>	レスポンスヘッダーの値を返します（大文字小文字を区別しない検索）。見つからない場合は <code>None</code> を返します。
<code>http_client_response_free</code>	<code>(resp: HttpClientResponse) -> Unit</code>	レスポンスと関連メモリを解放します。レスポンスの使用が終わったら呼び出してください。

クライアントの使用例

```
from http import http_get, http_post, http_client_status, http_client_body, http_client_header
```

シンプルな GET リクエスト

```
match http_get("http://example.com/api/data"):
    case Ok(resp):
        status = http_client_status(resp)
        body = http_client_body(resp)
        print(to_str(status) + ": " + body)
    case Err(e):
        print("Request failed")
```

ボディとヘッダー付きの POST リクエスト

```
headers: Map<str, str> = {"Content-Type": "application/json"}
match http_post("http://example.com/api/data", "{\"key\": \"value\"}", headers):
    case Ok(resp):
        print(http_client_body(resp))
    case Err(e):
        print("Request failed")
```

クライアントの動作仕様

- `http://` と `https://` の両方の URL をサポートします。HTTPS はシステム CA バンドル証明書検証による TLS を使用します。
- Host ヘッダーは URL に基づいて自動的に追加されます。
- `Connection: close` が常に送信されます。各リクエストは個別の TCP 接続を使用します。
- `Content-Length` は常に自動的に追加されます（空のボディには 0 を含む）。ユーザーが指定した `Content-Length` ヘッダーは正しい値で上書きされます。
- レスポンスボディの読み取りは `Content-Length`、`Transfer-Encoding: chunked`、または接続クローズまでの読み取りをサポートします。
- レスポンスヘッダーの検索は大文字小文字を区別しません。
- 接続タイムアウトは 5 秒、受信タイムアウトは 30 秒です。
- 接続失敗、無効な URL、不正なレスポンスの場合は `Err` を返します。
- `HttpClientResponse` は割り当てられたメモリ（ヘッダー、ボディ）を所有しています。メモリリークを避けるため、使用後に `http_client_response_free()` を呼び出してください。

リダイレクトの動作

HTTP クライアント関数はリダイレクトレスポンス（Location ヘッダー付きの 3xx）を自動的にフォローします。

- サポートされるステータスコード: 301、302、303、307、308
- 最大リダイレクト数: 10 (超過した場合は Err を返す)
- メソッド変換 (RFC 9110 準拠):
 - 301、302: POST は GET に変更される (ボディは破棄)。その他のメソッドは維持
 - 303: メソッドは常に GET に変更される (ボディは破棄)
 - 307、308: メソッドとボディは維持
- URL 解決: Location ヘッダーの絶対 URL、プロトコル相対 (//...), 絶対パス (/...), 相対パスをサポート
- ユーザー指定のヘッダーは各リダイレクトホップで再送信されますが、機密ヘッダー (Authorization、Proxy-Authorization、Cookie) はクロスオリジンリダイレクト (異なるホストまたはポート) ではストリップされます
- Location ヘッダーが欠落しているか空の場合、リダイレクトレスポンスがそのまま返されます
- 呼び出し元はすべてのリダイレクトをフォローした後の最終レスポンスのみを受け取ります

エラー処理

- `http_listen()` は `bind()` が失敗した場合 (例: ポートが既に使用中) にランタイムエラーを発生させます。
- 不正なリクエストやキープアライブ接続のアイドルタイムアウトにより接続が閉じられます。その後サーバーは新しい接続の受け入れを再開します。
- ハンドラ関数は常に `HttpResponse` を返す必要があります。デフォルトのレスポンスはありません。
- クライアント関数は `Result<HttpClientResponse, Error>` を返します。成功と失敗を処理するには `match` を使用してください。

[English](#) | [日本語](#) | [繁體中文](#)

I/O 関数リファレンス

標準入出力とファイル操作の関数一覧です。すべての関数は `io` からの明示的なインポートが必要です。

```
from io import read_text, write_text, file_exists
```

関数一覧

標準入力

関数	シグネチャ	説明
<code>read_line</code>	<code>() -> str</code>	stdin から 1 行読み取り (末尾改行除去)
<code>read_all</code>	<code>() -> str</code>	stdin を EOF まで全読み

ファイル I/O

関数	シグネチャ	説明
<code>read_text</code>	<code>(str) -> Result<str, Error></code>	ファイル全体を文字列として読み取り
<code>write_text</code>	<code>(str, str) -> Result<Unit, Error></code>	ファイルに文字列を書き込み (上書き)
<code>append_text</code>	<code>(str, str) -> Result<Unit, Error></code>	ファイル末尾に文字列を追記
<code>file_exists</code>	<code>(str) -> bool</code>	ファイル存在チェック
<code>delete_file</code>	<code>(str) -> Result<Unit, Error></code>	ファイル削除

関数	シグネチャ	説明
<code>read_bytes</code>	<code>(str) -> Result<List<u8>, Error></code>	ファイルをバイト列として読み取り
<code>write_bytes</code>	<code>(str, List<u8>) -> Result<Unit, Error></code>	バイト列をファイルに書き込み

バイト列変換

関数	シグネチャ	説明
<code>str_to_bytes</code>	<code>(str) -> List<u8></code>	文字列を UTF-8 バイト列に変換
<code>bytes_to_str</code>	<code>(List<u8>) -> Result<str, Error></code>	バイト列を文字列に変換

使用例

ファイルの読み書き

```
from io import read_text, write_text, append_text, file_exists, delete_file

match write_text("hello.txt", "Hello, World!"):
    case Ok(_):
        match read_text("hello.txt"):
            case Ok(content):
                print(content)    # Hello, World!
            case Err(e):
                print(e.message)
    case Err(e):
        print(e.message)

print(file_exists("hello.txt"))    # true

match delete_file("hello.txt"):
    case Ok(_):
        print(file_exists("hello.txt"))    # false
    case Err(e):
        print(e.message)
```

バイト操作

```
from io import str_to_bytes, bytes_to_str, write_bytes, read_bytes

bs = str_to_bytes("ABC")
print(length(bs))    # 3

match write_bytes("data.bin", bs):
    case Ok(_):
        match read_bytes("data.bin"):
            case Ok(rb):
                match bytes_to_str(rb):
                    case Ok(s):
                        print(s)    # ABC
                    case Err(e):
```

```

        print(e.message)
    case Err(e):
        print(e.message)
case Err(e):
    print(e.message)

```

標準入力からの読み取り

```

from io import read_line

name = read_line()
print(f"Hello, {name}!")

```

エラーハンドリング

ファイル操作は失敗時に終了するのではなく、`Result<T, Error>` を返します。`match` で `Ok/Err` パターンを使ってエラーを処理してください:

```

match read_text("missing.txt"):
    case Ok(content):
        print(content)
    case Err(e):
        print(e.message)    # cannot open file 'missing.txt' for reading

```

操作	エラー条件
<code>read_text</code> / <code>read_bytes</code>	ファイルが存在しない、または開けない
<code>write_text</code> / <code>write_bytes</code> / <code>append_text</code>	ファイルを書き込み用に開けない
<code>delete_file</code>	ファイルを削除できない
<code>bytes_to_str</code>	入力に NUL バイトが含まれている

備考

- `List<u8>` をバッファ型として使用します。標準的なリスト操作 (`length()`、`append()`、`slice()`、インデックスアクセス) がすべてバイトリストで使えます。
- ファイルパスは絶対パスを指定しない限り、カレントディレクトリからの相対パスとなります。
- `write_text` と `write_bytes` は既存ファイルを上書きします。既存ファイルに追記するには `append_text` を使用してください。

[English](#) | [日本語](#) | [繁體中文](#)

JSON 関数リファレンス

JSON のパースとシリアライズを行う関数一覧です。すべての関数は `json` からの明示的なインポートが必要です。

```

from json import parse, stringify, json_type, json_get, json_at, json_str, json_int, json_float, json_bo

```

概要

`json` パッケージは、JSON テキストをオペーク（不透明）な `JsonValue` 型にパースし、アクセサ関数で内容にアクセスし、テキストに再シリアライズする機能を提供します。JSON の値は異種型を含みうるため（オブジェクトには文字列、数値、真偽値、配列、ネストされたオブジェクトが含まれる）、オペークポインタ型と型付きアクセサ関数を使用します。

関数一覧

パース / シリアライズ

関数	シグネチャ	説明
parse	(str) -> Result<JsonValue, Error>	JSON 文字列を JsonValue にパース
stringify	(JsonValue) -> str	JsonValue をコンパクトな JSON テキストに変換
stringify	(JsonValue, int) -> str	インデント付きで整形出力 (引数はスペース数)

型クエリ

関数	シグネチャ	説明
json_type	(JsonValue) -> str	JSON の型を返す: "object", "array", "string", "number", "boolean", "null"

オブジェクト / 配列アクセス

関数	シグネチャ	説明
json_get	(JsonValue, str) -> Result<JsonValue, Error>	オブジェクトからキーでフィールドを取得
json_at	(JsonValue, int) -> Result<JsonValue, Error>	配列からインデックスで要素を取得

値の取り出し

関数	シグネチャ	説明
json_str	(JsonValue) -> Result<str, Error>	文字列値を取り出す
json_int	(JsonValue) -> Result<int, Error>	整数値を取り出す
json_float	(JsonValue) -> Result<float, Error>	浮動小数点値を取り出す
json_bool	(JsonValue) -> Result<bool, Error>	真偽値を取り出す

コレクション情報

関数	シグネチャ	説明
json_len	(JsonValue) -> int	配列の長さまたはオブジェクトのキー数を返す
json_keys	(JsonValue) -> List<str>	オブジェクトのキー一覧を返す

メモリ管理

関数	シグネチャ	説明
<code>json_free</code>	<code>(JsonValue) -> Unit</code>	JsonValue とその子要素をすべて解放

使用例

フィールドのパーズとアクセス

```
from json import parse, json_get, json_str, json_int, json_free

match parse("{\"name\": \"Alice\", \"age\": 30}"):
    case Ok(data):
        match json_get(data, "name"):
            case Ok(val):
                match json_str(val):
                    case Ok(name):
                        print(name)    # "Alice"
                    case Err(e):
                        print("error")
            case Err(e):
                print("error")
        json_free(data)
    case Err(e):
        print("parse error: " + e.message)
```

配列の操作

```
from json import parse, json_at, json_int, json_len, json_free

match parse("[10, 20, 30]"):
    case Ok(data):
        print(to_str(json_len(data)))    # 3
        match json_at(data, 0):
            case Ok(elem):
                match json_int(elem):
                    case Ok(n):
                        print(to_str(n))    # 10
                    case Err(e):
                        print("error")
            case Err(e):
                print("error")
        json_free(data)
    case Err(e):
        print("parse error")
```

整形出力付きシリアライズ

```
from json import parse, stringify, json_free

match parse("{\"key\": \"value\", \"count\": 42}"):
    case Ok(data):
        print(stringify(data, 2))
        # {
        #   "key": "value",
```

```

    # "count": 42
    # }
    json_free(data)
case Err(e):
    print("error")

```

注意事項

- `json_int` は整数と整数値の浮動小数点数（例: 42.0 → 42）の両方を受け付けます
- `json_float` は浮動小数点数と整数（例: 42 → 42.0）の両方を受け付けます
- `json_get` と `json_at` はパースツリー内の子要素への参照を返します。子要素に対して `json_free` を呼ばず、`parse` で返されたルート値に対してのみ呼んでください
- `json_type` は整数と浮動小数点数の両方に対して "number" を返します

[English](#) | [日本語](#) | [繁體中文](#)

数学関数 (math)

概要

`math` パッケージは数学定数と関数を提供します。`std` パッケージと異なり、自動インポートされません。明示的なインポートで使います。

```
from math import sqrt, PI, sin
```

定数

定数	型	説明
PI	float	円周率 (3.141592653589793)
E	float	ネイピア数 (2.718281828459045)
Inf	float	正の無限大
NaN	float	非数 (Not a Number)

```
from math import PI, E, Inf, NaN
```

```
circumference = 2.0 * PI * radius
```

絶対値

関数	シグネチャ	説明
<code>abs(x)</code>	<code>(int) -> int</code>	整数の絶対値
<code>abs(x)</code>	<code>(float) -> float</code>	浮動小数点数の絶対値

```
from math import abs
```

```
abs(-5)      # 5
abs(-3.14)   # 3.14
```

丸め

関数	シグネチャ	説明
<code>floor(x)</code>	<code>(float) -> int</code>	切り捨て
<code>ceil(x)</code>	<code>(float) -> int</code>	切り上げ
<code>round(x)</code>	<code>(float) -> int</code>	四捨五入

```
from math import floor, ceil, round
```

```
floor(3.7)    # 3
ceil(3.2)     # 4
round(3.5)    # 4
```

冪乗・平方根

関数	シグネチャ	説明
<code>sqrt(x)</code>	<code>(float) -> float</code>	平方根
<code>pow(x, y)</code>	<code>(float, float) -> float</code>	x の y 乗

```
from math import sqrt, pow
```

```
sqrt(9.0)     # 3.0
pow(2.0, 3.0)  # 8.0
```

対数

関数	シグネチャ	説明
<code>log(x)</code>	<code>(float) -> float</code>	自然対数 (底 e)
<code>log2(x)</code>	<code>(float) -> float</code>	底 2 の対数
<code>log10(x)</code>	<code>(float) -> float</code>	常用対数 (底 10)

指数

関数	シグネチャ	説明
<code>exp(x)</code>	<code>(float) -> float</code>	e の x 乗

三角関数

関数	シグネチャ	説明
<code>sin(x)</code>	<code>(float) -> float</code>	正弦 (x はラジアン)
<code>cos(x)</code>	<code>(float) -> float</code>	余弦 (x はラジアン)

関数	シグネチャ	説明
<code>tan(x)</code>	<code>(float) -> float</code>	正接 (x はラジアン)

逆三角関数

関数	シグネチャ	説明
<code>asin(x)</code>	<code>(float) -> float</code>	逆正弦 (結果はラジアン)
<code>acos(x)</code>	<code>(float) -> float</code>	逆余弦 (結果はラジアン)
<code>atan(x)</code>	<code>(float) -> float</code>	逆正接 (結果はラジアン)

2 引数関数

関数	シグネチャ	説明
<code>atan2(y, x)</code>	<code>(float, float) -> float</code>	y/x の逆正接 (結果はラジアン)
<code>hypot(x, y)</code>	<code>(float, float) -> float</code>	斜辺の長さ: $\sqrt{x^2 + y^2}$

判定関数

関数	シグネチャ	説明
<code>is_nan(x)</code>	<code>(float) -> bool</code>	x が NaN なら true
<code>is_inf(x)</code>	<code>(float) -> bool</code>	x が正または負の無限大なら true

```
from math import is_nan, is_inf, NaN, Inf
```

```
is_nan(NaN)    # true
is_inf(Inf)    # true
is_nan(1.0)    # false
```

[English](#)

ネットワーク (TCP) リファレンス

型

型	説明
<code>TcpListener</code>	TCP サーバーソケットの不透明ハンドル
<code>TcpStream</code>	TCP 接続の不透明ハンドル
<code>TlsStream</code>	TLS 暗号化された TCP 接続の不透明ハンドル

両方の型は不透明ポインタです。直接構築することはできません。`TcpListener/TcpStream` を取得するには `bind()` または `connect()` を使用し、`TlsStream` には `tls_connect()` を使用します。

関数 (net パッケージ)

これらの関数は明示的なインポートが必要です:

```
from net import bind, listen, accept, connect, listener_port, shutdown, set_timeout, set_recv_timeout, s
```

関数	シグネチャ	説明
bind	(host: str, port: int) -> Result<TcpListener, Error>	指定アドレスにバインドされた TCP サーバーソケットを作成します。失敗時は Err を返します。動的割り当てにはポート 0 を使用します。
listen	(listener: TcpListener, backlog: int) -> Result<Unit, Error>	受信接続のリスニングを開始します。失敗時は Err を返します。
accept	(listener: TcpListener) -> Result<TcpStream, Error>	新しい接続を受け入れます。クライアントの接続を最大 1 秒待ちます。タイムアウトまたは失敗時は Err を返します。
connect	(host: str, port: int) -> Result<TcpStream, Error>	リモート TCP サーバーに接続します。5 秒後にタイムアウトします。タイムアウトまたは失敗時は Err を返します。
listener_port	(listener: TcpListener) -> int	リスナーが実際にバインドされているポート番号を返します。ポート 0 (OS が割り当てるポート) でバインドした場合に便利です。
shutdown	(listener: TcpListener) -> Unit	リスナーに接続受け入れの停止を通知します。保留中の accept() は最大 1 秒以内に返されます。
tls_connect	(host: str, port: int) -> Result<TlsStream, Error>	TLS 暗号化でリモートサーバーに接続します。サーバー証明書をシステム CA バンドルに対して検証します。接続またはハンドシェイク失敗時は Err を返します。
set_timeout	(stream: TcpStream\ TlsStream, ms: int) -> Unit	受信と送信の両方のタイムアウトをミリ秒単位で設定します。
set_recv_timeout	(stream: TcpStream\ TlsStream, ms: int) -> Unit	受信タイムアウトをミリ秒単位で設定します。この時間内にデータが届かない場合、recv() は Err を返します。
set_send_timeout	(stream: TcpStream\ TlsStream, ms: int) -> Unit	送信タイムアウトをミリ秒単位で設定します。

組み込みオーバーロード関数

これらの関数は組み込みで、TCP ソケット型で動作します。インポートは不要です。

関数	シグネチャ	説明
send	(stream: TcpStream\ TlsStream, data: List<u8>) -> Result<int, Error>	TCP または TLS 接続を通じてバイトを送信します。成功時は送信バイト数を含む Ok を返し、失敗時は Err を返します。

関数	シグネチャ	説明
recv	(stream: TcpStream\ TlsStream, max: int) -> Result<List<u8>, Error>	最大 max バイトを受信します。接続クローズ時は空のリストを含む Ok を返し、エラー時は Err を返します。
close	(handle: TcpStream\ TlsStream) -> Unit	TCP または TLS ストリームを閉じます。
close	(handle: TcpListener) -> Unit	TCP リスナーを閉じます。

使用例

エコーサーバー

```

from net import bind, listen, accept, connect
from io import str_to_bytes, bytes_to_str

# サーバー
match bind("127.0.0.1", 8080):
    case Ok(server):
        match listen(server, 128):
            case Ok(_):
                match accept(server):
                    case Ok(conn):
                        match recv(conn, 4096):
                            case Ok(data):
                                match send(conn, data):
                                    case Ok(_):
                                        ...
                                    case Err(e):
                                        print(e.message)
                                case Err(e):
                                    print(e.message)
                            close(conn)
                        case Err(e):
                            ...
                    case Err(e):
                        print("listen failed")
            close(server)
        case Err(e):
            print("bind failed")

```

クライアント

```

match connect("127.0.0.1", 8080):
    case Ok(conn):
        match send(conn, str_to_bytes("hello")):
            case Ok(_):
                ...
            case Err(e):
                print(e.message)
        match recv(conn, 4096):
            case Ok(resp):

```

```

        match bytes_to_str(resp):
            case Ok(s):
                print(s)
            case Err(e):
                print(e.message)
        case Err(e):
            print(e.message)
    close(conn)
case Err(e):
    print("connect failed")

```

async fn による並行エコーサーバー

```

from net import bind, listen, accept, connect, listener_port
from io import str_to_bytes, bytes_to_str

```

```

async fn echo_server(server: TcpListener) -> str:
    match accept(server):
        case Ok(conn):
            match recv(conn, 4096):
                case Ok(data):
                    match send(conn, data):
                        case Ok(_):
                            ...
                        case Err(e):
                            ...
                    case Err(e):
                        ...
                close(conn)
            case Err(e):
                ...
    close(server)
    return "done"

```

```

match bind("127.0.0.1", 0):
    case Ok(server):
        match listen(server, 1):
            case Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                # ... port を使用するクライアントコード ...
                block_on(t)
            case Err(e):
                ...
        case Err(e):
            ...

```

タイムアウト設定

デフォルトでは、カスタムタイムアウトが設定されていない場合、`recv()` は 30 秒のタイムアウトを使用します。デフォルトを上書きするには `set_timeout()`、`set_recv_timeout()`、または `set_send_timeout()` を使用します:

```

from net import connect, set_recv_timeout

```

```

match connect("127.0.0.1", 8080):
    case Ok(conn):
        set_recv_timeout(conn, 5000) # 5 秒のタイムアウト
        match recv(conn, 4096):
            case Ok(data):
                ...
            case Err(e):
                print("timeout or error")
        close(conn)
    case Err(e):
        print("connect failed")

```

タイムアウトを無効にする（無期限に待機する）には 0 を渡します。

エラー処理

- `close()` を除くすべての TCP 関数は `Result<T, Error>` を返します。失敗を処理するには `Ok/Err` で `match` を使用してください。
- `recv()` はピアによって接続が閉じられた場合は空の `List<u8>` を含む `Ok` を返し、実際のエラー（タイムアウト、ソケットエラー）の場合は `Err` を返します。
- `close()` はソケットを閉じてハンドルを解放します。閉じた後のハンドルの使用は未定義動作です。

バイト変換

TCP 操作は `List<u8>` を使用します。文字列とバイトリストの変換には `io` パッケージの `str_to_bytes()` と `bytes_to_str()` を使用してください。

[English](#) | [日本語](#) | [繁體中文](#)

演算子リファレンス

優先順位テーブル

優先順位は数字が小さいほど高い（先に評価される）。

優先順位	演算子	説明	結合性
0	? !!	エラー伝播（後置）	左
1	()	グループ化	—
2	+x -x ~x	単項正・負、ビット NOT	右
3	**	累乗	右
3.5	as	型キャスト	左
4	* / % //	乗算・除算・剰余・整数除算	左
5	+ -	加算・減算	左
6	<< >> >>>	ビットシフト	左
7	&	ビット AND	左
8	^	ビット XOR	左
9	\	ビット OR	左
10	== != < <= > >= in not in	比較・所属	左
11	not	論理 NOT	右
12	and	論理 AND	左
13	or	論理 OR	左
13.5	??	null 合体	左
14	?:	三項条件	右

算術演算子

演算子	説明	例
+	加算 / 文字列結合	1 + 2 -> 3、"a" + "b" -> "ab"
-	減算	5 - 3 -> 2
*	乗算 / 文字列繰り返し	4 * 3 -> 12、"ab" * 3 -> "ababab"
/	除算 (常に float)	7 / 2 -> 3.5
//	整数除算 (-∞ 方向への切り捨て)	7 // 2 -> 3、-7 // 2 -> -4
%	剰余	7 % 3 -> 1
**	累乗 (常に float)	2 ** 10 -> 1024.0
-x	単項マイナス	-5, -3.14
+x	単項プラス	+5 (符号変更なし)

```
a = 10 // 3      # 3 (int)
b = 10 / 3       # 3.3333... (float)
c = 2 ** 8       # 256.0 (float)
s = "foo" + "bar" # "foobar"
```

比較演算子

すべて bool を返す。

演算子	説明
==	等しい
!=	等しくない
<	より小さい
<=	以下
>	より大きい
>=	以上

- 数値型 (int / float) と bool に対して使用可能。
- str 同士は辞書順 (バイト順) で比較。
- レコード型はフィールドごとの自動比較で == と != をサポート ([構造体リファレンス](#)参照)。
- in 演算子はセット、リスト、マップに対する所属チェックに使用 (x in s)。
- not in 演算子は in の否定 (x not in s)。
- マップの場合、in はキーの存在を確認します。

```
x = 3 < 5      # true
y = "abc" < "abd" # true (辞書順)
s = {1, 2, 3}
z = 2 in s     # true
w = 4 not in s # true
xs = [1, 2, 3]
a = 2 in xs    # true (リスト線形探索)
m = {"a": 1}
b = "a" in m   # true (マップキー検索)
```

論理演算子

演算子	説明	型
and	論理 AND	bool x bool -> bool
or	論理 OR	bool x bool -> bool
not	論理 NOT	bool -> bool

```
a = true and false    # false
b = true or false     # true
c = not true          # false
```

ビット演算子

int 型のみ使用可能。float や bool に適用するとコンパイルエラー。

演算子	説明	例
&	ビット AND	0b1100 & 0b1010 -> 0b1000
\	ビット OR	0b1100 \ 0b1010 -> 0b1110
^	ビット XOR	0b1100 ^ 0b1010 -> 0b0110
~	ビット NOT (単項)	~0 -> -1
<<	左シフト	1 << 4 -> 16
>>	算術右シフト	16 >> 2 -> 4
>>>	論理右シフト	-1 >>> 1 -> 9223372036854775807

```
flags = 0b0001 | 0b0010    # 3
masked = flags & 0b0011    # 3
shifted = 1 << 8           # 256
```

エラー伝播演算子 (? / !!)

後置 ? 演算子は Result 値をアンラップします。値が Ok(v) の場合は v に評価されます。値が Err(e) の場合は、外側の関数から即座に Err(e) を返します。

!! 演算子は ? のエイリアスで、同一のセマンティクスを持ちます。両方を互換的に使用できます。

外側の関数は Result 型を返す必要があります。

```
fn safe_divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

fn compute(a: int, b: int, c: int) -> Result<int, Error>:
    x = safe_divide(a, b)?    # b == 0 の場合は Err を早期リターン
    y = safe_divide(x, c)!!
    return Ok(y + 1)
```

これは以下の match パターンと等価ですが、はるかに簡潔です:

```
fn compute(a: int, b: int, c: int) -> Result<int, Error>:
    match safe_divide(a, b):
        case Ok(x):
            match safe_divide(x, c):
                case Ok(y):
                    return Ok(y + 1)
                case Err(e):
                    return Err(e)
```

```
case Err(e):
    return Err(e)
```

三項条件演算子

```
x = condition ? true_value : false_value
```

condition を評価し、真であれば true_value を、偽であれば false_value を返します。両方の分岐は同じ型でなければなりません。右結合なので、ネストされた三項演算子は右から左に結合されます。

```
x = 3 > 2 ? 10 : 20      # 10
s = false ? "yes" : "no" # "no"
```

ネスト（右結合）

```
y = true ? (false ? 1 : 2) : 3 # 2
```

範囲演算子

.. 演算子は両端を含む整数の範囲を生成します。

```
xs = 1 .. 5      # [1, 2, 3, 4, 5]
```

```
for i in 1 .. 3:
    print(i)      # 1 2 3
```

結果は左オペランドから右オペランドまで（両端含む）のすべての整数を含む List<int> です。

null 合体演算子 (??)

```
x = option_val ?? default_val
```

option_val が Some(v) の場合は v を返します。それ以外の場合は default_val を返します。右辺のオペランドは Option の内部型と同じ型でなければなりません。

```
a: int? = Some(10)
b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0
```

複合代入演算子

変数を更新するショートハンド。x op= y は x = x op y と等価。

演算子	等価な式
x += y	x = x + y
x -= y	x = x - y
x *= y	x = x * y
x /= y	x = x / y
x %= y	x = x % y
x //= y	x = x // y

演算子	等価な式
<code>x **= y</code>	<code>x = x ** y</code>
<code>x &= y</code>	<code>x = x & y</code>
<code>x \ = y</code>	<code>x = x \ y</code>
<code>x ^= y</code>	<code>x = x ^ y</code>
<code>x <<= y</code>	<code>x = x << y</code>
<code>x >>= y</code>	<code>x = x >> y</code>

```
x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 3    # x = 8
x &= 6    # x = 0
```

インクリメント・デクリメント演算子

変数を 1 増減させるポストフィックス演算子。ステートメントとしてのみ使用可能。内部的にはそれぞれ `x = x + 1`、`x = x - 1` にデシュガーされる。

演算子	等価な式
<code>x++</code>	<code>x = x + 1</code>
<code>x--</code>	<code>x = x - 1</code>

```
count = 0
count++    # count = 1
count++    # count = 2
count--    # count = 1
```

```
f = 1.5
f++        # f = 2.5 (int 1 が float に型昇格)
```

注意: `++` / `--` はステートメントとしてのみ使用でき、式の中では使えません。`@const` 変数にはインクリメント・デクリメントを適用できません（不変性が強制されます）。

演算の型規則

演算	左辺型	右辺型	結果型
<code>+</code>	int	int	int
<code>-</code>	float	int / float	float
<code>*</code>	int	float	float
<code>/</code>	任意の数値	任意の数値	float
<code>//</code>	int	int	int
<code>//</code>	float または int (片方 float)	-	float
<code>**</code>	任意の数値	任意の数値	float
<code>%</code>	int	int	int
<code>%</code>	float または int (片方 float)	-	float
<code>+</code>	str	str	str
<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	数値 / bool / str	同型	bool
<code>*</code>	str	int	str

演算	左辺型	右辺型	結果型
in	任意	Set / List / Map<K, V>	bool
not in	任意	Set / List / Map<K, V>	bool
& \ ^ ~ << >> >>>	int	int	int
and or not	bool	bool	bool

演算子オーバーロード

ユーザー定義型に対して演算子の動作を定義できます。

構文

二項演算子 (引数 2 個)

```
fn operator+(a: MyType, b: MyType) -> MyType:
    ...
```

単項演算子 (引数 1 個)

```
fn operator-(a: MyType) -> MyType:
    ...
```

オーバーロード可能な演算子一覧

種別	演算子
算術 (二項)	+ - * / % ** //
比較 (二項)	== != < <= > >=
ビット (二項)	& \ ^ << >> >>>
論理 (二項)	and or
所属	in
添字	[] (読み取り)、[] = (書き込み)
呼び出し	()
キャスト	as
単項	- ~ not
複合代入	+= -= *= /= %= // = ** = &= \ = ^= <<= >>=

戻り値型の制約

比較演算子と論理演算子は bool を返す必要があります:

種別	演算子	必須戻り値型
比較	== != < <= > >=	bool
論理	and or not	bool
所属	in	bool
キャスト	as	必須 (ターゲット型)

OK

```
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

エラー: comparison operator '==' must return 'bool', but returns 'int'

```
fn operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算術演算子・ビット演算子には戻り値型の制約はありません。

二項 / 単項の区別

引数の個数で区別します。

```
# 二項 -
fn operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)

# 単項 -
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

複合代入演算子のオーバーロード

複合代入演算子（+=、-= 等）は個別にオーバーロードできます。大きなデータ構造のインプレース最適化を可能にします。

```
record Matrix:
    data: List
    rows: int
    cols: int

fn operator+=(a: Matrix, b: Matrix) -> Matrix:
    for i in range(len(a.data)):
        a.data[i] = a.data[i] + b.data[i]
    return a
```

解決優先順位 $x += y$ が評価される際:

1. `operator+=` がその型に定義されている場合 → 直接呼び出す
2. `operator+=` が未定義で `operator+` がある場合 → $x = x + y$ にフォールバック
3. どちらも未定義の場合（組み込み型以外）→ コンパイルエラー

```
record Vec2:
    x: float
    y: float

fn operator+=(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

v = Vec2(1.0, 2.0)
v += Vec2(3.0, 4.0) # operator+= を直接呼び出す
# v.x == 4.0, v.y == 6.0
```

複合代入演算子は引数が正確に 2 つ必要で、戻り値型の制約はありません。

添字演算子のオーバーロード

`[]`（読み取り）と `[] =`（書き込み）演算子でユーザー定義型にカスタムの添字アクセスを定義できます。複数インデックスアクセス（例: `m[row, col]`）もサポートされています。

```
record Grid:
    a: int
    b: int
    c: int
    d: int

# 読み取り: 2 個以上のパラメータ（オブジェクト + インデックス）が必要
```

```
fn operator[] (g: Grid, row: int, col: int) -> int:
    if row == 0 and col == 0:
        return g.a
    if row == 0 and col == 1:
        return g.b
    if row == 1 and col == 0:
        return g.c
    return g.d
```

書き込み: 3個以上のパラメータ (オブジェクト + インデックス + 値) が必要

```
fn operator[]=(g: Grid, row: int, col: int, value: int):
    ...
```

```
g = Grid(1, 2, 3, 4)
print(g[0, 1])      # 2
g[1, 0] = 99
```

ユーザー定義の添字演算子が最初に試行され、一致しない場合は組み込みの添字動作 (リスト、マップ、配列) がフォールバックとして使用されます。

所属演算子のオーバーロード

in 演算子をオーバーロードしてカスタムの所属チェックを定義できます。bool を返す必要があります。

```
record Range:
    lo: int
    hi: int
```

```
fn operator in(value: int, r: Range) -> bool:
    return value >= r.lo and value < r.hi
```

```
r = Range(1, 10)
print(5 in r)      # true
print(15 not in r) # true
```

ユーザー定義の in 演算子が最初に試行され、一致しない場合は組み込みの動作 (セット、マップ、リスト) がフォールバックとして使用されます。in が定義されていれば not in は自動的にサポートされます。

呼び出し演算子のオーバーロード

() 演算子でレコードを呼び出し可能なオブジェクトとして動作させることができます。2 個以上のパラメータ (オブジェクト + 引数) が必要です。

```
record Adder:
    base: int
```

```
fn operator()(a: Adder, x: int) -> int:
    return a.base + x
```

```
add5 = Adder(5)
print(add5(10))    # 15
```

レコード値を保持する変数を関数のように呼び出すと、コンパイラはまず operator() のオーバーロードを試みます。一致しない場合は、他の呼び出し解決戦略 (関数、コンストラクタ、ラムダ) が優先されます。

キャスト演算子のオーバーロード

`as` 演算子をオーバーロードしてカスタムの型変換を定義できます。パラメータは正確に 1 つ（ソース値）で、戻り値型（ターゲット型）の指定が必要です。ソース型と戻り値型でディスパッチされます。

```
record Celsius:
  value: int

record Fahrenheit:
  value: int

fn operator as(c: Celsius) -> Fahrenheit:
  return Fahrenheit(c.value * 9 // 5 + 32)
```

```
c = Celsius(100)
f = c as Fahrenheit  # Fahrenheit(212)
```

ユーザー定義の `as` 演算子が最初に試行され、一致しない場合は組み込みのキャスト（`int`、`float`、`bool`、`str` 等）がフォールバックとして使用されます。

[English](#) | [日本語](#) | [繁體中文](#)

パッケージリファレンス

概要

Ry はパッケージシステムでコードを管理します。パッケージは単一の `.ry` ファイル、またはディレクトリ（複数の `.ry` ファイルを含む）のいずれかです。 `from` 文でパッケージをインポートします。

`std` パッケージ（標準ライブラリ）はすべてのプログラムに自動的にインポートされます。

インポート構文

全定義インポート

```
from math
```

パッケージ内のすべての関数・型をインポートします。

選択インポート

```
from math import sqrt
```

指定した定義のみをインポートします。

複数選択インポート

```
from math import sqrt, PI
```

カンマ区切りで複数の定義を選択インポートします。

パッケージ解決

ドット区切りのパッケージ名は以下のように解決されます:

インポート文	解決先
<code>from math</code>	<code>math/</code> ディレクトリ (パッケージ) または <code>math.ry</code> ファイル
<code>from utils.math</code>	<code>utils/math/</code> ディレクトリまたは <code>utils/math.ry</code> ファイル
<code>from str</code>	<code>str/</code> ディレクトリまたは <code>str.ry</code> ファイル

解決順序

各検索パスに対して: 1. ディレクトリ (`{path}/`) — 存在すればディレクトリ内の全 `.ry` ファイルを読み込む (パッケージ) 2. ファイル (`{path}.ry`) — 単一ファイル (後方互換)

ディレクトリパッケージ

パッケージがディレクトリに解決された場合: - ディレクトリ内のすべての `.ry` ファイルが自動的に読み込まれる - `_` で始まるファイルは除外される - 特別なエントリファイル (`__init__.py` のようなもの) は不要 - ディレクトリ内のファイルで定義されたすべての関数・型がエクスポートされる

プライベートシンボル

名前が `_` (アンダースコア) で始まる定義はパッケージ内部のプライベートシンボルとして扱われ、インポートできません:

- ワイルドカードインポート (`from pkg`) では `_` プレフィックスのシンボルが自動的に除外される
- 名前指定インポート (`from pkg import _helper`) はコンパイルエラーになる

```
# mylib/internal.ry
fn _helper() -> int:      # プライベート — インポート不可
    return 42
fn public_api() -> int:   # パブリック — インポート可能
    return _helper()

mypackage/
  calc.ry      # fn add(), fn sub()
  string.ry    # fn concat()

from mypackage      # add, sub, concat をインポート
from mypackage import add # add のみインポート
```

標準ライブラリ (std)

`std` パッケージはすべてのプログラムに自動的にインポートされます。提供する機能: - 組み込み関数 (`print`, `length`, `range` など) - 文字列関数 (`contains`, `find`, `replace` など) - 型変換関数 (`to_int`, `to_float`, `to_str`) - コレクション関数 (`map`, `filter`, `sort` など)

サブパッケージ

以下のサブパッケージは明示的なインポートが必要です:

パッケージ	説明
<code>math</code>	数学定数・関数
<code>io</code>	ファイル I/O・標準入力・バイト変換

```
from math import sqrt, PI, sin
```

標準ライブラリのパッケージから特定の定義を明示的にインポートすることもできます:

```
from str import contains
```

RY_HOME

標準ライブラリは \$RY_HOME/lib/std/ にインストールされます。RY_HOME のデフォルト値は ~/.ry です。

```
export RY_HOME="$HOME/.ry" # デフォルト
```

RY_ENV

RY_ENV 環境変数でランタイム環境モードを制御します。--env=<value> CLI フラグでも指定可能です。

値	エイリアス	.env 読み込み	lib 探索
prod	production	無効	リポジトリビルド用プロジェクトオーバーライド → \$RY_HOME/lib → exe/./lib → exe/lib
dev	development	.env.dev → .env	prod と同じ
test	—	.env.test → .env	prod と同じ
staging	—	.env.staging → .env	prod と同じ
internal	—	.env.internal → .env	リポジトリビルド用プロジェクトオーバーライド → exe/./lib → exe/lib (\$RY_HOME スキップ)
(未設定) (デフォルト)	—	.env のみ	prod と同じ

エイリアスは自動的に正規形に解決されます。例えば RY_ENV=production は prod に正規化されます。

prod モードではセキュリティのため .env ファイルを読み込みません。本番環境の秘密情報はインフラレベルの環境変数管理 (CI/CD、シークレットマネージャー等) で管理してください。

その他のモードでは .env.<環境名> を先に読み込み (存在する場合)、次に .env を読み込みます。既存の環境変数は上書きされないため、環境別の値が優先されます。

短縮形 (推奨)

```
RY_ENV=dev ./build/ry app.ry
```

フルネーム (後方互換)

```
RY_ENV=development ./build/ry app.ry
```

CLI フラグ

```
./build/ry --env=dev test
```

prod モード: .env は読み込まれない

```
RY_ENV=prod ./build/ry app.ry
```

Ry 自体の開発時の追加 isolation

```
RY_ENV=internal ./build/ry test
```

Ry のソースツリー内でビルドされた ry 実行バイナリは、プロジェクトの `package.toml` からリポジトリローカルの `stdlib` オーバーライドを使用できます。これにより、`~/.ry/lib/std` が古い場合でも、チェックアウトされた `lib/std` とリポジトリビルドの整合性が保たれます。インストール済みの ry バイナリはこのオーバーライドを無視し、`$RY_HOME/lib/std` を引き続き使用します。

検索パスの優先順位

1. インポート元ファイルのディレクトリ
2. 現在の Ry チェックアウトからのリポジトリローカル `stdlib` オーバーライド（リポジトリビルドの ry 使用時）
3. `$RY_HOME/lib`（標準ライブラリの場合）
4. 実行ファイル相対の `lib/` ディレクトリ
5. `RY_PATH` 環境変数に含まれるパス（コロン区切り）

RY_PATH 環境変数

`RY_PATH` にコロン区切りでディレクトリを指定すると、パッケージ検索パスに追加されます。

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

制約

制約	詳細
使用可能な位置	トップレベルのみ（関数・ブロック内は不可）
二重インポート	自動でスキップ（エラーにならない）
循環インポート	コンパイルエラー

```
# エラー例：ブロック内でのインポート
fn main():
    from math    # エラー：トップレベル以外ではインポート不可

# OK: 同じパッケージを複数回インポートしてもエラーにならない
from math
from math    # スキップされる
```

パッケージファイルの作成

単一ファイルパッケージ

```
# calc.ry
fn add(a: int, b: int) -> int:
    return a + b

fn sub(a: int, b: int) -> int:
    return a - b

# main.ry
from calc import add, sub
```



```
print(add(1, 2))    # 3
print(sub(5, 3))   # 2
```

ディレクトリパッケージ

```
mylib/
  calc.ry
  string.ry
# main.ry
from mylib import add, concat
```

[English](#) | [日本語](#) | [繁體中文](#)

プロジェクト管理

CLI 概要

<code>ry <file.ry> [args...]</code>	<code># Ry スクリプトを実行</code>
<code>echo '<code>' ry</code>	<code># 標準入力からコードを実行</code>
<code>ry test [options] [<file> <dir>]</code>	<code># テストを実行</code>
<code>ry init</code>	<code># プロジェクトを初期化</code>
<code>ry new <project-name></code>	<code># 新規プロジェクトを作成</code>
<code>ry fmt [options] [<file> <dir>]</code>	<code># ソースファイルをフォーマット</code>
<code>ry self-update [options]</code>	<code># ry 自身を更新</code>

グローバルオプション

オプション	説明
<code>-h, --help</code>	ヘルプを表示
<code>-v, --version</code>	バージョンを表示
<code>--env=<env></code>	環境を設定 (production development internal)。 RY_ENV 環境変数を上書きする。

標準入力からの実行

ファイル引数が指定されず、標準入力ターミナルでない場合、ry は標準入力からソースコードを読み取って実行します:

```
echo 'print("hello")' | ry
```

ry init - プロジェクト初期化

カレントディレクトリを Ry プロジェクトとして初期化します。

```
ry init
```

生成されるファイル・ディレクトリ

```
my-project/
  package.toml      # プロジェクト設定ファイル
  src/
    main.ry         # エントリーポイント (サンプルコード)
```

動作

1. `package.toml` が既に存在する場合はエラー終了する
 2. `src/` ディレクトリを作成（存在しなければ）
 3. `package.toml` を生成（`name` はカレントディレクトリ名）
 4. `src/main.ry` を生成（既に存在する場合はスキップ）
-

ry new - 新規プロジェクト作成

新しいディレクトリを作成し、Ry プロジェクトとして初期化します。

```
ry new my-project
```

生成されるファイル・ディレクトリ

```
my-project/
  package.toml      # プロジェクト設定ファイル
  src/
    main.ry         # エントリポイント（サンプルコード）
```

動作

1. プロジェクト名が指定されていない場合はエラー終了する
 2. 同名のディレクトリが既に存在する場合はエラー終了する
 3. `<project-name>/` ディレクトリを作成
 4. その中に `src/` ディレクトリを作成
 5. `package.toml` を生成（`name` は指定されたプロジェクト名）
 6. `src/main.ry` を生成
-

ry fmt - コードフォーマッター

`.ry` ソースファイルを一貫した 2 スペースインデントと正規スタイルでフォーマットします。

```
ry fmt          # プロジェクト内の全 .ry ファイルをフォーマット (package.toml 必要)
ry fmt src/main.ry # 単一ファイルをフォーマット
ry fmt src/      # ディレクトリ内の全 .ry ファイルを再帰的にフォーマット
ry fmt --check   # フォーマット済みか確認 (未フォーマットなら exit 1)
ry fmt --check src/ # 特定ディレクトリを確認
```

フォーマットルール

- ブロックレベルごとに 2 スペースインデント
- 二項演算子の前後にスペース (`a + b`、`a+b` ではない)
- カンマの後にスペース (`f(a, b)`、`f(a,b)` ではない)
- トップレベル定義間（関数、レコード、列挙型）に空行
- コメントは保持される

動作

1. ソースファイルを読み取り、AST にパースし、正規フォーマットで再出力する
2. フォーマット結果をファイルに書き戻す（インブレース）
3. `--check` 指定時は未フォーマットファイルを報告し、存在する場合はコード 1 で終了（CI 向け）
4. 再帰フォーマット時は `.git/`、`build/`、`node_modules/` ディレクトリをスキップ

注意事項

- LLVM の初期化は不要（高速起動）
- 複合代入演算子（+=、-= 等）はパーサーがデシュガーするため、フォーマット後は展開形（`x = x + expr`）になる
- 16 進数（0xFF）・2 進数（0b1010）リテラルは 10 進数表記に変換される

ry test - テスト実行

テストファイル（*.test.ry）を探索して実行します。テスト構文の詳細は[テスト](#)を参照。

```
ry test                # *.test.ry ファイルを自動検出して全テスト実行
ry test tests/spec     # ディレクトリ内の全テストを実行
ry test test_file.ry   # 特定のテストファイルを実行
ry test -p             # テストを並列実行
ry test -w             # ウォッチモード: ファイル変更時に再実行
ry test --coverage     # 行カバレッジ情報を収集
```

オプション

オプション	説明
-p, --parallel	テストを並列実行
-w, --watch	ファイル変更を監視して再実行
--coverage, --cov	カバレッジ情報を収集
-h, --help	ヘルプを表示

動作

1. 引数なしの場合、package.toml を検索してプロジェクトルートを見つけ、*.test.ry ファイルを再帰的に探索する（.git、build、node_modules はスキップ）
2. 全テスト成功なら終了コード 0、失敗があれば 1
3. --coverage と --parallel を同時に指定すると順次実行にフォールバック

ry self-update - セルフアップデート

ry 自身を最新バージョンに更新します。GitHub Releases からバイナリをダウンロードし、現在の実行ファイルを置換します。

```
ry self-update         # 最新安定版に更新
ry self-update --nightly # 最新 nightly プレリリースに更新
ry self-update v0.0.1  # 指定バージョンに更新
```

動作

1. 現在のバージョンを表示
2. 引数に応じて更新対象のバージョンを解決
 - 引数なし: GitHub の最新安定リリース（/releases/latest）
 - --nightly: 最新のプレリリース
 - バージョン指定: 指定されたタグのリリース
3. 現在のバージョンと同じ場合は "Already up to date." で終了
4. バイナリをダウンロードして現在の実行ファイルを置換

注意事項

- 実行には `curl` および `tar` コマンドが必要です
- パーミッション不足でバイナリの置換に失敗した場合、`sudo` の使用を促すメッセージが表示されます (`sudo` は自動的に実行されません)
- ダウンロードはテンポラリディレクトリに対して行われますが、ファイルシステムをまたぐ `cp` フォールバックが中断された場合、バイナリが不完全な状態になる可能性があります

package.toml 設定ファイル

プロジェクトのメタデータとパス設定を TOML 形式で記述します。

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"
```

```
[paths]
src = "src"
```

[project] セクション

キー	説明
name	プロジェクト名（初期化時はディレクトリ名）
version	バージョン文字列
entry	エントリポイントとなるソースファイル

[paths] セクション

キー	説明
src	ソースコードディレクトリ

TOML サブセット仕様

package.toml は以下の TOML サブセットをサポートします。

- セクションヘッダ: `[section]`
- キー・値ペア: `key = "value"`（文字列値のみ）
- コメント: `#` から行末まで
- 空行は無視される

[English](#) | [日本語](#) | [繁體中文](#)

正規表現関数リファレンス

正規表現関数の一覧です。すべての関数は UFCS 記法をサポートしています。パターン文字列は標準的な正規表現構文を使用します。

注意: Phase 1 ではパターンは通常の文字列として渡します。専用の正規表現リテラル構文は将来追加される可能性があります。

関数一覧

関数	シグネチャ	説明
<code>regex_match</code>	<code>(str, str) -> bool</code>	テキスト全体がパターンにマッチするか
<code>regex_search</code>	<code>(str, str) -> int</code>	最初のマッチの開始位置を返す（見つからない場合 -1）
<code>regex_replace</code>	<code>(str, str, str) -> str</code>	マッチした部分を置換文字列で置換
<code>regex_split</code>	<code>(str, str) -> List<str></code>	パターンマッチで分割
<code>regex_find_all</code>	<code>(str, str) -> List<str></code>	重複しないすべてのマッチを返す

サポートするパターン構文

構文	説明	例
<code>abc</code>	リテラル文字	"hello"
<code>.</code>	任意の 1 文字（改行を除く）	"a.c" は "abc", "aXc" にマッチ
<code> </code>	選択	"cat dog" は "cat" か "dog" にマッチ
<code>*</code>	0 回以上の繰り返し	"a*" は "", "a", "aaa" にマッチ
<code>+</code>	1 回以上の繰り返し	"a+" は "a", "aaa" にマッチ
<code>?</code>	0 回または 1 回	"a?" は "" か "a" にマッチ
<code>{n}</code>	ちょうど n 回	"a{3}" は "aaa" にマッチ
<code>{n,m}</code>	n 回以上 m 回以下	"a{2,4}" は "aa" ~ "aaaa" にマッチ
<code>{n,}</code>	n 回以上	"a{2,}" は "aa", "aaa", ... にマッチ
<code>*?</code>	0 回以上（非貪欲）	".*?" は最短マッチ
<code>+?</code>	1 回以上（非貪欲）	".+?" は最短マッチ
<code>??</code>	0 回または 1 回（非貪欲）	"a??" は 0 回を優先
<code>{n,m}?</code>	範囲（非貪欲）	"a{2,4}?" は n 回を優先
<code>(...)</code>	グループ	"(ab)+" は "abab" にマッチ
<code>[abc]</code>	文字クラス	"[aeiou]" は母音にマッチ
<code>[a-z]</code>	文字範囲	"[a-z]+" は小文字の単語にマッチ
<code>[^...]</code>	否定文字クラス	"[^0-9]" は数字以外にマッチ
<code>^</code>	文字列先頭アンカー	"^hello"
<code>\$</code>	文字列末尾アンカー	"world\$"
<code>\d</code>	数字 [0-9]	"\d+" は数値にマッチ
<code>\D</code>	数字以外 [^0-9]	
<code>\w</code>	単語文字 [a-zA-Z0-9_]	"\w+" は識別子にマッチ
<code>\W</code>	単語文字以外	
<code>\s</code>	空白文字	"\s+" はスペースやタブにマッチ
<code>\S</code>	空白文字以外	
<code>\b</code>	単語境界	"\bword\b" は単語全体にマッチ

構文	説明	例
<code>\B</code>	非単語境界	<code>"\Bword"</code> は単語の内部にマッチ
<code>(?i)</code>	大文字小文字無視フラグ	<code>"(?i)hello"</code> は <code>"HELLO"</code> にマッチ
<code>\.</code>	エスケープされた特殊文字	<code>"\."</code> はリテラルの <code>.</code> にマッチ

使用例

`regex_match`

```
print(regex_match("[a-z]+", "hello")) # true
print(regex_match("[0-9]+", "hello")) # false
print(regex_match("[a-zA-Z_]\w*", "my_var")) # true
```

`regex_search`

```
pos = regex_search("[0-9]+", "abc123def")
print(pos) # 3
```

`regex_replace`

```
s = regex_replace("[0-9]+", "a1b2c3", "X")
print(s) # aXbXcX
```

`regex_split`

```
parts = regex_split("\\s+", "hello world foo")
print(length(parts)) # 3
print(parts[0]) # hello
```

`regex_find_all`

```
matches = regex_find_all("[0-9]+", "a1b23c456")
print(length(matches)) # 3
print(matches[0]) # 1
print(matches[1]) # 23
```

範囲量指定子

```
print(regex_match("\\d{3}-\\d{4}", "123-4567")) # true
print(regex_match("a{2,4}", "aaa")) # true
print(regex_match("(ab){2,}", "ababab")) # true
```

非貪欲（最短）マッチ

```
# 貪欲: 最長マッチ
g = regex_replace("\\.*\\\"", "\\\"a\\\" and \\\"b\\\"", "X")
print(g) # X
```

```
# 非貪欲: 最短マッチ
l = regex_replace("\\.*?\\\"", "\\\"a\\\" and \\\"b\\\"", "X")
print(l) # X and X
```

```
# 個別の HTML タグを取得
```

```
tags = regex_find_all("<.*?>", "<a> <bb> <ccc>")
print(length(tags)) # 3
```

注意: 非貪欲マッチはマッチ全体の長さを制御します。グループ（括弧で囲んだ部分式）がない場合、greedy/lazy の混在パターンは PCRE エンジンと異なる動作をする場合があります。

単語境界

```
# 単語全体にマッチ
pos = regex_search("\\bworld\\b", "hello world")
print(pos) # 6

# すべての単語を取得
words = regex_find_all("\\b\\w+\\b", "hello world foo")
print(length(words)) # 3

# \\B は非境界（単語の内部）にマッチ
pos2 = regex_search("\\Bworld", "helloworld")
print(pos2) # 5
```

大文字小文字を無視したマッチ

```
# (?i) をパターンの先頭に置くと大文字小文字を無視
print(regex_match("(?i)hello", "HELLO")) # true
print(regex_match("(?i)hello", "Hello")) # true

# 文字クラスでも動作
print(regex_match("(?i)[a-z]+", "ABC")) # true

# replace や find_all でも使用可能
s = regex_replace("(?i)hello", "Hello HELLO hello", "X")
print(s) # X X X
```

注意: (?i) はパターンの先頭に記述する必要があり、パターン全体に適用されます。部分的な大文字小文字無視（例: (?i:sub)pattern）はサポートされていません。

UFCS 記法

```
# pattern.function(text, ...)
m = "[a-z]+".regex_match("hello")
print(m) # true
```

[English](#) | [日本語](#) | [繁體中文](#)

構造体リファレンス

概要

構造体はスタック上の値型です。record キーワードで定義します。構造体には invariant 節で契約による設計の不変条件を定義できます。[契約による設計](#)を参照。

命名規則: 構造体名は PascalCase（例: Point、Rectangle）を使用する必要があります。フィールド名は snake_case を使用します。コンパイラがこれらの規則を強制します。

定義構文

```
record 型名:  
    フィールド名: 型  
    フィールド名: 型
```

例

```
record Point:  
    x: int  
    y: int  
  
record Rectangle:  
    width: float  
    height: float
```

コンストラクタ

フィールド定義順に引数を渡します。名前付き引数はサポートされていません。

```
p = Point(10, 20)  
r = Rectangle(3.0, 4.5)
```

フィールドアクセス

ドット記法でフィールドを読み取ります。

```
p = Point(10, 20)  
print(p.x)    # 10  
print(p.y)    # 20
```

フィールド代入

変数宣言	フィールド代入
可変 (@const なし)	可能
@const	コンパイルエラー

```
p = Point(10, 20)  
p.x = 100    # OK: 可変変数  
  
@const  
q = Point(10, 20)  
q.x = 100    # エラー: @const 変数のフィールドは変更不可
```

関数引数・戻り値としての使用

```
fn distance(p: Point) -> float:  
    return (p.x * p.x + p.y * p.y) as float
```



```
fn make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

ネスト構造体

構造体を別の構造体のフィールドとして使用できます。

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

比較 (== / !=)

レコード型は == と != 演算子を自動的にサポートします。フィールドごとの比較（構造的等価性）が行われます。

```
record Point:
    x: int
    y: int

p1 = Point(10, 20)
p2 = Point(10, 20)
p3 = Point(30, 40)

print(p1 == p2)    # true
print(p1 != p3)    # true
```

- すべてのフィールドが順番に比較されます。== ではすべてのフィールドが等しい必要があり、!= では少なくとも1つのフィールドが異なる必要があります。
 - ネストされたレコードは再帰的に比較されます。
 - ユーザー定義の operator== または operator!= がある場合、自動生成版より優先されます。
-

レコードのサブタイプ化（継承）

レコードは < 構文を使った単一継承をサポートします。子レコードは親のすべてのフィールドを継承します。

構文

```
record 子の名前 < 親の名前:
    子固有のフィールド: 型
```

例

```
record HttpError < Error:
    status: int
```

```
url: str
```

フィールドの継承

- 子レコードはレイアウトの先頭に親のすべてのフィールドを継承します。
- コンストラクタは親のフィールドを先に、次に子固有のフィールドを取ります。

```
err = HttpError("not found", 404, 404, "/api")
print(err.message) # "not found" (Error から継承)
print(err.status)  # 404 (固有フィールド)
```

サブタイプの型強制

子の値は親の型が期待される場所に渡すことができます。子は自動的にスライスされ、親のプレフィックスフィールドが抽出されます（値型スライシング）。

```
fn handle(e: Error) -> str:
    return e.message

err = HttpError("fail", 500, 500, "/api")
handle(err) # OK — HttpError が Error に型強制される
```

深い継承

レコードは継承チェーンを形成できます。各レベルはすべての祖先フィールドを継承します。

```
record DetailedHttpError < HttpError:
    detail: str

# コンストラクタ: Error フィールド + HttpError フィールド + 固有フィールド
derr = DetailedHttpError("fail", 500, 500, "/x", "server crash")
handle(derr) # OK — Error (祖父母) に型強制される
```

ルール

ルール	詳細
単一継承のみ	record A < B: — 親は1つのみ
深い継承	record C < B: where record B < A: — 許可
名前の衝突	子のフィールドが親のフィールドと同名 → コンパイルエラー
自動 == / to_str	継承したフィールドもすべて含む
不変条件の継承	親の invariant: 節は子レコードの構築・変更時にもチェックされる
サブタイプの型強制	適用先: 関数引数、return、Err()、フィールド代入、? 演算子
ジェネリックバウンド	<T: RecordName> で型パラメータをレコードのサブタイプに制約
@const	継承したフィールドも含むすべてのフィールドに適用

制約とエラー

制約	詳細
同一フィールド名の重複	コンパイルエラー
@const 変数のフィールド代入	コンパイルエラー
print に構造体を直接渡す	コンパイルエラー (print 非対応)

エラー例: 同一フィールド名の重複

```
record Bad:
  x: int
  x: int    # エラー
```

エラー例: print に構造体を渡す

```
p = Point(1, 2)
print(p)    # エラー
```

列挙型 (enum)

概要

列挙型は名前付き定数の集合です。デフォルトでは i64 整数 (0, 1, 2, ...) の連番として表現されます。明示的な整数値を割り当てることもできます。

定義構文

```
enum 型名:
  バリエーション名
  バリエーション名
  ...
```

例

```
enum Color:
  Red
  Green
  Blue
```

バリエーションアクセス

:: 演算子でバリエーションにアクセスします。

```
c = Color::Red
print(c)    # Red
```

比較

enum 値は整数なので == / != でそのまま比較できます。

```
print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true
```

if 文での使用

```
c = Color::Green
if c == Color::Red:
    print("red")
elif c == Color::Green:
```

```

    print("green")
else:
    print("blue")

```

関数引数

型名として enum 名を使用します。

```

fn is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false

```

print

print() でバリエーション名が出力されます。

```

c = Color::Blue
print(c)    # Blue

```

明示的な値の割り当て

simple enum のバリエーションに明示的な整数値を割り当てることができます。HTTP ステータスコードやビットマスクパターンなどに有用です。

```

enum HttpStatus:
    Ok = 200
    NotFound = 404
    InternalError = 500

s = HttpStatus::NotFound
print(s)                                # NotFound
print(s == HttpStatus::NotFound)        # true

enum Permission:
    Read = 1
    Write = 2
    Execute = 4

```

ルール: - simple enum (関連データを持たない ADT バリエーションなし) のみ対応- 値は整数リテラルのみ (負の値も可) - いずれかのバリエーションに明示的な値がある場合、全バリエーションに必要 (混在不可) - 重複値はコンパイラエラー - print() はバリエーション名を表示 (整数値ではない)

ADT バリエーションの名前付きフィールド

ADT バリエーションのフィールドにはドキュメント目的で名前をオプションで付けることができます。名前付きフィールドは定義を自己説明的にしますが、構築やパターンマッチングのセマンティクスは変わりません。

```

enum Shape:
    Circle(radius: float)
    Rect(width: float, height: float)
    Point

```

- 構築は常に位置ベース: Shape::Circle(3.14) であり、Shape::Circle(radius: 3.14) ではない。
- パターンマッチングはユーザーが選んだ変数名で束縛: case Shape::Circle(r):。
- フィールド名は snake_case でなければならない。単一バリエーション内での名前付きと名前なしの混在は不可。
- 名前なし構文 (Circle(float)) は引き続き有効。

制約とエラー

制約	詳細
バリエーションアクセスは <code>EnumName::VariantName</code> バリエーション値	<code>::</code> 演算子が必須 デフォルトは自動割り当て (0, 1, 2, ...)、 <code>=</code> 値で明示的に指定可能
比較は整数比較 名前付きフィールド名	<code>==</code> , <code>!=</code> が使用可能 <code>snake_case</code> でなければならない; バリエーション内で重複不可; 名前付きと名前なしの混在不可

[English](#) | [日本語](#) | [繁體中文](#)

テスト機能

Ry は RSpec 風のテスト構文を内蔵しています。ry test サブコマンドでテストファイルを実行します。

実行方法

```
ry test                # プロジェクト内の *.test.ry を自動検出して実行
ry test tests/spec     # 指定ディレクトリ以下の *.test.ry を再帰的に実行
ry test test_file.ry   # 特定のテストファイルを実行
ry test -p             # 全テストを並列実行 (-p または --parallel)
ry test -p tests/      # 指定ディレクトリのテストを並列実行
ry test -w             # ウォッチモード: ファイル変更時にテストを自動再実行 (-w または --watch)
ry test -w -p          # ウォッチモード + 並列実行
ry test -w tests/      # 指定ディレクトリをウォッチ
ry test --coverage     # 全テストをラインカバレッジ付きで実行
ry test --cov          # --coverage の短縮形
```

終了コードは全テスト成功時に 0、失敗がある場合は 1 です。

自動検出モード

ry test を引数なしで実行すると:

1. package.toml を探してプロジェクトルートを特定
2. プロジェクトルート以下の *.test.ry ファイルを再帰的に検出 (.git、build、node_modules はスキップ)
3. 各ファイルを実行し、結果を集計

構文

describe / it

```
describe("説明文", fn():
  it("テストケース名", fn():
    # テスト本体
    expect(実際の値).to_eq(期待値)
  )
)
```

- describe と it は説明文字列とラムダ引数 fn(): を第二引数に取る

- `describe` ブロック内には `it` ブロックやその他の文（変数宣言など）を記述可能
- 各 `it` ブロックは独立したテストケース
- `describe / expect` は `ry test` でのみ使用可能（通常の `ry` 実行ではコンパイルエラー）

トレイリングブロック構文

任意の関数呼び出し（`describe/it/mock` を除く）にトレイリングブロック構文が使えます。() の後に `:` を付けると、インデントブロックが引数なしラムダとして最後の引数に渡されます:

```
# 以下は等価:
foo("arg"):
    bar()

foo("arg", fn():
    bar()
)
```

`expect / マッチャー`

マッチャー	説明	対応型
<code>to_eq(expected)</code>	等値比較	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_not_eq(expected)</code>	等しくないこと	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_be_true()</code>	<code>true</code> であること	<code>bool</code>
<code>to_be_false()</code>	<code>false</code> であること	<code>bool</code>
<code>to_be_none()</code>	<code>None</code> であること	<code>Option</code>
<code>to_be_some()</code>	<code>Option</code> が <code>Some</code> であること	<code>Option</code>
<code>to_be_ok()</code>	<code>Result</code> が <code>Ok</code> であること	<code>Result</code>
<code>to_be_err()</code>	<code>Result</code> が <code>Err</code> であること	<code>Result</code>
<code>to_contain(val)</code>	コンテナが値を含むこと	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_not_contain(val)</code>	コンテナが値を含まないこと	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_be_greater_than(v)</code>	<code>actual > v</code> であること	<code>int</code> , <code>float</code>
<code>to_be_less_than(v)</code>	<code>actual < v</code> であること	<code>int</code> , <code>float</code>
<code>to_be_greater_than_or_eq(v)</code>	<code>actual >= v</code> であること	<code>int</code> , <code>float</code>
<code>to_be_less_than_or_eq(v)</code>	<code>actual <= v</code> であること	<code>int</code> , <code>float</code>
<code>to_have_length(n)</code>	長さが <code>n</code> であること	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_be_empty()</code>	長さが <code>0</code> であること	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_start_with(prefix)</code>	文字列が <code>prefix</code> で始まること	<code>str</code>
<code>to_end_with(suffix)</code>	文字列が <code>suffix</code> で終わること	<code>str</code>

`fail`

現在のテストを即座に失敗としてマークします。

```
it("should not reach here", fn():
    fail("unexpected error")
)
```

- `fail()` — 汎用メッセージでテストを失敗にする
- `fail(msg)` — カスタムメッセージでテストを失敗にする
- `fail()` 後も実行は継続される（テストを中断はしない）
- `ry test` モードでのみ使用可能

出力形式

Calculator

```
+ adds numbers
+ subtracts
- fails test (赤色)
  line 10: expected 3, got 2
```

2 passed, 1 failed

- + は成功（緑色）、- は失敗（赤色）
 - 失敗時は行番号と期待値/実際の値を表示
-

例

```
describe("Arithmetic", fn():
  it("adds integers", fn():
    expect(1 + 2).to_eq(3)

  )
  it("compares strings", fn():
    expect("hello").to_eq("hello")

  )
  it("checks booleans", fn():
    expect(3 > 1).to_be_true()

  )
)
describe("Booleans", fn():
  it("false check", fn():
    expect(1 > 2).to_be_false()

  )
)
```

モック

mock(fn_name, replacement)

現在の it ブロック内で関数をモック実装に差し替えます。it ブロック終了時にモックは自動的にクリアされます。

```
fn fetch_data() -> str:
  return "real data"

describe("mocking", fn():
  it("replaces function", fn():
    mock(fetch_data, fn() => "fake")
    expect(fetch_data()).to_eq("fake")

  )
  it("auto-restores", fn():
    expect(fetch_data()).to_eq("real data")
```

```
)  
)
```

- 第1引数は関数名（識別子、文字列ではない）
- 第2引数は差し替え用ラムダ
- 差し替え関数は元の関数と同じ引数型・戻り値型である必要がある
- `it` ブロック終了時にモックは自動復元される

`verify(fn_name)`

モック済み関数の呼び出し回数を `int` で返します。

```
describe("verify", fn():  
  it("counts calls", fn():  
    mock(fetch_data, fn() => "fake")  
    fetch_data()  
    fetch_data()  
    expect(verify(fetch_data)).to_eq(2)  
  )  
)
```

モックの制限事項

- オーバーロードされた関数のモックは非対応
 - キャプチャ付きクロージャでのモックは非対応（プレーンラムダのみ）
 - `@native fn` のモックは非対応
-

パラメタライズドテスト (`@each`)

`@each` は同じテストを複数のパラメータセットで実行します。タプルのリストを指定して `it` ブロックに適用します:

```
@each([  
  (1, 2, 3),  
  (0, 0, 0),  
  (-1, 1, 0)  
)  
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):  
  expect(a + b).to_eq(expected)  
)
```

- リストにはラムダのパラメータ数と同じアリティのタプルを含める
 - 説明文の `{0}`, `{1}`, ... はパラメータ値で置換される
 - 各タプルは独立したテストケースとして実行される
 - 対応するパラメータ型: `int`, `float`, `bool`, `str`
-

プロパティベーステスト (`@property`)

`@property` はランダムな入力を生成し、テストを複数回実行します:

```
@property(count=100)  
it("addition is commutative", fn(a: int, b: int):  
  expect(a + b).to_eq(b + a)  
)
```


- `count=N` でランダム試行回数を指定 (デフォルト: 100)
- 失敗時は反例 (失敗した入力値) が表示される
- 最初の失敗でテストを停止
- 対応するパラメータ型: `int` `([-1000, 1000])`、`float` `([-1000.0, 1000.0])`、`bool`、`str` (ランダム ASCII、0-20 文字)

テストカバレッジ

`--coverage` (または `--cov`) フラグでラインカバレッジを計測できます:

```
ry test --coverage          # 全テスト + カバレッジサマリー
ry test --cov tests/spec/math.test.ry # 単一ファイル
ry test --coverage tests/spec/      # ディレクトリ
```

出力例

```
Test Coverage Summary:
  tests/spec/math.test.ry    100.0% (74/74 lines)
  tests/spec/strings.test.ry 92.3% (24/26 lines)
-----
Total                        95.1% (98/100 lines)
```

- 標準ライブラリのファイルは除外され、ユーザーコードのみが対象
- `--coverage` と `--parallel` を同時に指定した場合、逐次実行にフォールバック

制限事項

- `describe` のネストは未対応
- `before_each` / `after_each` は未対応

[English](#) | [日本語](#) | [繁體中文](#)

型リファレンス

型一覧

型	内部表現	リテラル例	説明
<code>int</code>	<code>i64</code>	<code>42, -7, 0xFF, 0b1010</code>	64 ビット符号付き整数
<code>u8</code>	<code>i8</code>	(専用リテラルなし)	符号なし 8 ビット整数 (0-255)。型アノテーション <code>b: u8 = 42</code> で使用
<code>float</code>	<code>f64</code>	<code>3.14, 0.5</code>	64 ビット浮動小数点数
<code>bool</code>	<code>i1</code>	<code>true, false</code>	真偽値
<code>str</code>	<code>ptr</code>	<code>"hello", "", "a\nb"</code>	文字列 (ヒープ上の不変バイト列)
<code>Unit</code>	<code>void</code>	(戻り値なし)	戻り値のない関数の戻り値型。→ <code>Unit</code> で明示的に指定する必要がある
<code>Option<T></code>	<code>{ i1, T }</code>	<code>Some(42), None</code>	値が存在するかもしれない型
<code>(T1, T2, ...)</code>	LLVM StructType (literal)	<code>(1, 3.14)</code>	タプル型

型	内部表現	リテラル例	説明
List<T>	ptr (ヒープ)	[1, 2, 3]	動的配列
Map<K, V>	ptr (ヒープ)	{"a": 1}	ハッシュマップ
Set<T>	ptr (ヒープ)	{1, 2, 3}	重複なしの集合
fn(T1, T2) -> R	ptr (関数ポインタ)	fn(x: int) => x * 2	関数型
ユーザー定義型	LLVM StructType (named)	record Point: ...	record キーワードで定義する構造体
enum	i64 / タグ付きユニオン	Color::Red, Shape::Circle(3.14)	enum キーワードで定義する列挙型 (関連データをサポート)
Error	{ ptr, i64 }	Error("msg"), Error("msg", 404)	組み込みエラー型
any	{ i64, [8 x i8] }	x: any = 42	任意のプリミティブ値を保持できるタグ付きユニオン
T1 \ T2	{ i64, [N x i8] }	int \ str	union 型 (複数の型のいずれかを保持)
int リテラル	i64	42, 0 \ 1	int リテラル型 (値の制約)
str リテラル	ptr	"N" \ "S"	str リテラル型 (値の制約)
範囲	i64	1..12, -10..10	範囲型 (整数の範囲制約)
i8	i8	x: i8 = 42, x = 42i8	8 ビット符号付き整数 (低レベル、暗黙の変換なし)
i16	i16	x: i16 = 100, x = 100i16	16 ビット符号付き整数 (低レベル、暗黙の変換なし)
i32	i32	x: i32 = 42, x = 42i32	32 ビット符号付き整数 (低レベル、暗黙の変換なし)
i64	i64	x: i64 = 100, x = 100i64	64 ビット符号付き整数 (低レベル、暗黙の変換なし)
u8	i8	x: u8 = 200, x = 200u8	8 ビット符号なし整数 (低レベル、暗黙の変換なし)
u16	i16	x: u16 = 60000, x = 60000u16	16 ビット符号なし整数 (低レベル、暗黙の変換なし)
u32	i32	x: u32 = 3000000000, x = 100u32	32 ビット符号なし整数 (低レベル、暗黙の変換なし)
u64	i64	x: u64 = 100, x = 100u64	64 ビット符号なし整数 (低レベル、暗黙の変換なし)
f32	float	x: f32 = 3.14, x = 3.14f32	32 ビット浮動小数点数 (低レベル、暗黙の変換なし)
[T; N]	[N x T]	buf: [i32; 8]	固定長配列。低レベル型 T の N 要素 (スタック割り当て、連続メモリ)

型アノテーション構文

変数宣言時に型を明示できます。型が推論可能な場合は省略可能です。

```
x: int = 42
b: u8 = 255
f: float = 3.14
s: str = "hello"
b: bool = true
opt: Option<int> = Some(10)
t: (int, float) = (1, 3.14)
xs: List<int> = [1, 2, 3]
m: Map<str, int> = {"a": 1}
s: Set<int> = {1, 2, 3}
fn_val: fn(int) -> int = fn(x: int) => x * 2
u: int | str = 42
a: any = 42
```

使用可能な型名一覧

型名	備考
int	組み込みスカラー型
u8	組み込みスカラー型 (符号なし 0-255)
float	組み込みスカラー型
bool	組み込みスカラー型
str	組み込み文字列型
Unit	戻り値なし関数の戻り値型
Option<T>	ジェネリック型 (T は任意の型)
(T1, T2, ...)	タプル型 (要素数・型の組み合わせは任意)
List<T>	ジェネリック動的配列型
Map<K, V>	ジェネリックハッシュマップ型
Set<T>	ジェネリック集合型
fn(T1, ...) -> R	関数型
Error	組み込みエラー型 (message: str, code: int)
any	任意のプリミティブ値 (int, float, bool, str) または Unit を保持できる組み込み型。暗黙の変換をサポート: 具体型の値は any への代入時に自動的にラップされ、any の値は具体型への代入時にランタイム型チェック付きで自動アンラップされる。any(int) → float の自動昇格に対応。詳細は any 型 を参照
T1 \ T2 \ ...	union 型 (\ で区切った複数の型のいずれか)
i8	低レベル 8 ビット符号付き整数 (暗黙の変換なし)
i16	低レベル 16 ビット符号付き整数 (暗黙の変換なし)
i32	低レベル 32 ビット符号付き整数 (暗黙の変換なし)
i64	低レベル 64 ビット符号付き整数 (暗黙の変換なし)
u8	低レベル 8 ビット符号なし整数 (暗黙の変換なし)
u16	低レベル 16 ビット符号なし整数 (暗黙の変換なし)
u32	低レベル 32 ビット符号なし整数 (暗黙の変換なし)
u64	低レベル 64 ビット符号なし整数 (暗黙の変換なし)
f32	低レベル 32 ビット浮動小数点数 (暗黙の変換なし)
[T; N]	低レベル型 T の N 要素の固定長配列。スタック割り当て、連続メモリ。インデックスの読み書きと length() をサポート
ユーザー定義型名	record または enum キーワードで宣言した型

型エイリアス

`type` キーワードで既存の型に新しい名前を付けます。エイリアスは元の型と完全に互換性があります。

```
type Meters = float
type StringList = List<str>
```

```
d: Meters = 3.14
names: StringList = ["Alice", "Bob"]
```

命名規則: 型エイリアス名は PascalCase (例: `Meters`、`StringList`) を使用する必要があります。コンパイラがこの規則を強制します。

型エイリアスは関数型、リテラル型、範囲型にも使用できます:

```
type Callback = fn(int, int) -> int

add: Callback = fn(a: int, b: int) => a + b
print(add(3, 4))    # 7

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

m: Month = 6
d: Direction = "N"
n: Digit = 5
```

リテラル型

リテラル型は、変数の値を特定の定数値に制限します。定数値の場合はコンパイル時に、動的な値の場合は実行時に制約がチェックされます。

int リテラル型

```
x: 42 = 42           # 単一リテラル型
y: 0 | 1 = 0         # int リテラルの union
z: 0 | 1 = 0
z = 1                # OK
# z = 2              # コンパイルエラー (定数) または実行時エラー (動的値)
```

str リテラル型

```
dir: "N" | "S" | "E" | "W" = "N"
# @const bad: "N" | "S" = "X"    # コンパイルエラー
```

制約チェック

- **コンパイル時:** 代入値が定数 (`ConstantInt` や文字列リテラル) の場合、コンパイル時にチェックされ違反時はコンパイルエラー
 - **実行時:** 値が動的 (関数の戻り値など) の場合、実行時にチェックされ違反時はプログラムがエラー終了
-

範囲型

範囲型は、整数変数の値を連続した範囲 (両端を含む) に制限します。

```
month: 1..12 = 6          # OK
# @const bad: 1..12 = 0    # コンパイルエラー: 範囲外
# @const bad: 1..12 = 13   # コンパイルエラー: 範囲外

t: -10..10 = -5          # 負の範囲もサポート
```

可変変数での再代入（実行時チェック）

```
x: 1..12 = 6
x = 12          # OK
# x = dynamic_value() # 実行時チェック: 範囲外ならエラー終了
```

関数パラメータでの使用

```
fn set_month(m: 1..12) -> int:
    return m

set_month(6)          # OK
# set_month(13)       # コンパイルエラー（定数引数）
```

none キーワードと Option 型の省略記法

none キーワードは Option 型の値が存在しないことを表し、None と同等です。

T? 構文は Option<T> の省略記法です。

```
x: int? = 42          # Option<int> と同等
y: int? = none        # None と同等
```

```
fn find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none
```

F-String（文字列補間）

f"..." 構文による文字列補間です。{} 内の式が評価され、文字列に変換されます。

```
name = "world"
print(f"Hello {name}")    # Hello world

a = 1
b = 2
print(f"{a} + {b} = {a + b}") # 1 + 2 = 3
```

補間で使用可能な型

{ } 内には int、float、bool、str に評価される任意の式を使用できます。

エスケープシーケンス

シーケンス	出力
{ {	{ (リテラルの波括弧)
} }	} (リテラルの波括弧)
\n \r \t \\ \"	通常の文字列と同じ

```
print(f"{{{braces}}}") # {braces}
```

型キャスト (as)

as キーワードによる明示的な型変換です。

```
x = 42 as float      # 42.0
y = 3.14 as int      # 3
z = 1 as bool        # true
s = 42 as str        # "42"
b = 255 as u8        # u8 値 255
```

サポートされるキャスト

変換元	変換先	動作
int	float	SIToFP
float	int	切り捨て (FPToSI)
int	bool	0 -> false、非 0 -> true
bool	int	false -> 0、true -> 1
int / float / bool	str	文字列表現
int	u8	切り捨て (下位 8 ビット)
u8	int	ゼロ拡張

```
int | i8 / i16 / i32 / i64 | 切り捨て (i64 の場合は恒等) |
i8 / i16 / i32 / i64 | int | 符号拡張 (SExt) |
int | u8 / u16 / u32 / u64 | 切り捨て (u64 の場合は恒等) |
u8 / u16 / u32 / u64 | int | ゼロ拡張 (ZExt) |
符号付き | 符号付き (より広い) | 符号拡張 (SExt) |
符号付き | 符号付き (より狭い) | 切り捨て |
符号なし | 符号なし/符号付き (より広い) | ゼロ拡張 (ZExt) |
符号なし | 符号なし/符号付き (より狭い) | 切り捨て |
符号付き / 符号なし整数 | float | SIToFP / UIToFP -> f64 |
float | 符号付き / 符号なし整数 | FPToSI / FPToUI |
float | f32 | FPTrunc |
f32 | float | FPExt |
符号付き整数 | f32 | SIToFP |
符号なし整数 | f32 | UIToFP |
f32 | 符号付き / 符号なし整数 | FPToSI / FPToUI |
```

サポートされないキャスト (例: `str as int`) はコンパイルエラーになります。文字列から数値への変換には `to_int()` / `to_float()` を使用してください。

関連データを持つ enum (ADT)

バリエーション名の後ろに括弧で型を指定することで、enum バリエーションに関連データを持たせることができます。括弧なしのバリエーションは従来通りの単純なタグとして機能します。

```
enum Shape:
  Circle(float)
  Rectangle(float, float)
  Point
```

名前付きフィールド

バリエーションにはドキュメント明確化のために名前付きフィールドをオプションで使用できます。名前付きフィールドはバリエーション定義を自己説明的にしますが、ランタイム動作は変わりません。構築とパターンマッチングは位置ベースのままです。

```
enum Shape:
  Circle(radius: float)
  Rectangle(width: float, height: float)
  Point
```

ルール: - フィールド名は `snake_case` でなければなりません。- 単一バリエーション内では、すべてのフィールドが名前付きか名前なしのいずれかでなければなりません（混在不可）。- バリエーション内のフィールド名の重複はコンパイルエラーです。

コンストラクタ

`EnumName::Variant(value)` の構文でデータ付きバリエーションを構築します。フィールドが名前付きであっても引数は常に位置ベースです。

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```

バインディング付きパターンマッチング

`case EnumName::Variant(binding):` の形式で関連データを取り出せます。バインディングにはフィールド名ではなくユーザーが選択した変数名を使用します。

```
match c:
  case Shape::Circle(r):
    print(r)           # 3.14
  case Shape::Rectangle(w, h):
    print(w)
    print(h)
  case Shape::Point:
    print("point")
```

内部表現

ADT `enum` はタグ付きユニオンとして格納されます: `{ i64 tag, [N x i8] data }`。N は最大バリエーションのペイロードに合わせたサイズです。

ジェネリック enum

`enum` は `<T>` の形式で型パラメータを持てます。これにより、同じ `enum` 構造で異なる型のペイロードを保持できます。

```
enum MyOption<T>:
  MySome(T)
  MyNone
```

使用法

コンパイラが型を推論できない場合は、具体的な型引数を指定してインスタンス化します。

```
a = MyOption<int>::MySome(42)
b = MyOption<int>::MyNone
```

```
match a:
  case MyOption::MySome(v):
    print(v)          # 42
  case MyOption::MyNone:
    print("none")
```

Error 型

エラーハンドリング用の組み込み型です。Error は message (str) と code (int) の 2 つのフィールドを持ちます。

```
e = Error("something went wrong")      # code のデフォルトは 0
e2 = Error("not found", 404)           # 明示的な code
```

```
print(e.message)    # something went wrong
print(e2.code)       # 404
print(e2)            # Error: not found (code: 404)
```

Result を使ったエラーハンドリング

失敗する可能性のある関数は Result<V, E> を返します:

```
fn divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)
```

```
match divide(10, 2):
  case Ok(v):
    print(v)          # 5
  case Err(e):
    print(e.message)
```

戻り値が意味を持たない場合は Result<Unit, Error> を使用します:

```
fn save(path: str, data: str) -> Result<Unit, Error>:
  return Ok(0 as u8)    # Unit プレースホルダー
```

```
match save("/tmp/test.txt", "hello"):
  case Ok(_):
    print("saved")
  case Err(e):
    print(e.message)
```

Result 型

Result<V, E> は 2 つのコンストラクタを持つ組み込みパラメータ化型です:

- Ok(value) – 成功バリエーション
- Err(error) – エラーバリエーション

match を使った網羅的なエラーハンドリングに使用します。Ok と Err の両方のケースをカバーするか、_ ワイルドカードを使用する必要があります。

テストマッチャー: - expect(x).to_be_ok() - 結果が Ok であることをアサート - expect(x).to_be_err() - 結果が Err であることをアサート

内部表現

Error は { ptr message, i64 code } として表現されます。Result<V, E> は { i1 isOk, V okValue, E errValue } として表現されます。

union 型

| を使って複数の型を持ちうる変数を宣言できます。

```
x: int | str = 42
x = "hello"      # 再代入可能 (union のいずれかの型)
print(x)         # hello
```

関数引数・戻り値での使用

```
fn show(x: int | str) -> int:
    print(x)
    return 0

fn get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

内部表現

union 型は { i64 tag, [N x i8] data } として表現されます。tag は各コンポーネント型のインデックス (アルファベット順ソート後) を示し、data は最大コンポーネントサイズ分のバイト配列です。

制約

- union に含まれない型を代入するとコンパイルエラー
- int | str と str | int は同じ型 (正規化される)
- print() で union 値を出力すると、実行時のタグに基づいて適切な型で表示される

any 型

any 型は、任意のプリミティブ値を保持できる組み込みの動的型です。Python の柔軟な型付けアプローチに倣い、静的な型の保証が不要な場面で、ジェネリクスや union 型を使わずに複数の型を扱えるようにします。

保持可能な型

型	タグ	説明
int	0	64 ビット符号付き整数
float	1	64 ビット浮動小数点数
bool	2	真偽値
str	3	文字列
Unit	4	Unit 値 (戻り値なし関数用)

`any` にはコレクション型 (`List`、`Map`、`Set`)、リソース型 (`TcpListener`、`TcpStream` 等)、関数ポインタ、ユーザー定義型 (`record`、`enum`) は保持できません。

内部表現

`any` はタグ付きユニオンとして実装されています:

```
{ i64 tag, [8 x i8] data } // 合計 16 バイト
```

`tag` フィールドが格納されている型を識別し、`data` フィールドに値 (最大 8 バイト) を保持します。

ラッピングとアンラッピング

具体型の値は `any` への代入時に自動的にラップされ、`any` の値は具体型への代入時に自動的にアンラップされます。

```
# ラッピング: 具体型 → any
x: any = 42          # int が any にラップされる
x = "hello"         # 異なる型への再代入が可能

# アンラッピング: any → 具体型
fn get_value() -> any:
    return 42
n: int = get_value() # any(int) が int にアンラップされる

# アンラップ時の int → float 自動昇格
f: float = get_value() # any(int) がアンラップされ float に昇格
```

実行時の型がターゲット型と一致しない場合 (例: `any(str)` を `int` 変数にアンラップ)、ランタイムエラーが発生します。

再代入

`any` 変数は、保持可能な任意の型の値に再代入できます:

```
x: any = 42
x = 3.14      # OK: float を保持
x = "hello"   # OK: str を保持
x = true     # OK: bool を保持
```

算術演算

両方のオペランドが `any` の場合、実行時の型に基づいて演算がディスパッチされます:

演算	型	結果
+	int + int	int
+	float + float	float
+	int + float	float
+	str + str	str (連結)
-	数値	int または float
*	数値	int または float
*	str * int / int * str	str (繰り返し)
/	数値	float (常に)
//	int // int	int
//	float を含む場合	float
%	数値	int または float
**	数値	float (常に)

演算	型	結果
単項 -	int	int
単項 -	float	float

一方が `any` で他方が具体型の場合、具体型の値は演算前に自動的にラップされます。

```
x: any = 10
y: any = x + 20    # 20 が自動ラップされる; 結果は any(int) = 30
```

互換性のない型の組み合わせ（例: `str - int`）はランタイムエラーになります。

比較演算

演算	動作
<code>==</code> , <code>!=</code>	同じ型同士で動作; int/float の混合比較が可能
<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	数値 (int/float 混合可) と文字列 (辞書順)

```
x: any = 3
y: any = 3.0
print(x == y)    # true (int/float 比較)
```

比較時の型不一致（例: `int < str`）はランタイムエラーになります。

文字列変換

`any` の値は `print()` と f-string 補間をサポートします:

```
x: any = 42
print(x)          # 42
print(f"value: {x}") # value: 42
```

変換ルール: `int` → 10 進文字列、`float` → `%g` 形式、`bool` → `"true"/"false"`、`str` → そのまま、`Unit` → `"Unit"`。

型付き関数への any の受け渡し

`any` の値を具体的な引数型を持つ関数に渡せます。値は実行時の型チェック付きで自動的にアンラップされます:

```
fn add_one(x: int) -> int:
    return x + 1

v: any = 42
result = add_one(v)    # any(int) が int にアンラップされる; 結果は 43
```

型規則（演算時の型変換）

演算	左辺	右辺	結果型	備考
<code>+</code> <code>-</code> <code>*</code>	int	int	int	
<code>+</code> <code>-</code> <code>*</code>	u8	u8	u8	低レベル型: native 幅の unsigned 演算、暗黙の昇格なし

演算	左辺	右辺	結果型	備考
+ - *	float または int	float または int (片方が float)	float	暗黙の float 昇格
/	任意の数値	任意の数値	float	常に float
//	任意の数値	任意の数値	int または float	切り捨て除算 (-∞ 方向); int オペランド同士なら int、片方が float なら float
**	任意の数値	任意の数値	float	libm pow 使用
%	int	int	int	
%	float または int	float または int (片方が float)	float	
+	str	str	str	文字列結合
== != < <= > >=	str	str	bool	辞書順比較
== != < <= > >=	数値または bool	数値または bool	bool	
in	任意	Set	bool	要素がセットに含まれるか
& \ ^ ~ << >>	int	int	int	float にはエラー
+ - *	i32	i32	i32	低レベル型: 暗黙の変換なし、同じ型が必要
/ //	i32	i32	i32	符号付き整数除算 (SDiv)
/ //	u32	u32	u32	符号なし整数除算 (UDiv)
%	i32	i32	i32	符号付き剰余 (SRem)
%	u32	u32	u32	符号なし剰余 (URem)
+ - * /	f32	f32	f32	
== !=	i32/u32	i32/u32	bool	符号非依存の等値比較
< <= > >=	i32	i32	bool	符号付き比較 (ICMP_SLT 等)
< <= > >=	u32	u32	bool	符号なし比較 (ICMP_ULT 等)
>>	i32	i32	i32	算術右シフト (符号保持)
>>	u32	u32	u32	論理右シフト (ゼロ埋め)
**	低レベル	任意	エラー	低レベル型には累乗演算子は未対応
混合	低レベル	異なる型	エラー	低レベル型と高レベル型の混合はコンパイルエラー

エスケープシーケンス (str リテラル内)

シーケンス	意味
\n	改行
\r	復帰

シーケンス	意味
\t	タブ
\\	バックスラッシュ
\"	ダブルクォート
\0	ヌル文字

型安全性の制約

- **暗黙の拡大変換** –関数呼び出しでは安全な拡大変換がサポートされます: `u8 → int`、`u8 → float`、`int → float`。二項演算子では `int` と `float` の混合で `float` 昇格が発生します。`u8` は native 幅の unsigned 演算を行う低レベル型であり、二項演算子での `u8` と `int` の混合はコンパイルエラーです。縮小変換 (例: `float → int`) は暗黙には許可されません。`int` リテラルから `u8` への縮小変換は型アノテーション `b: u8 = 42` でのみ許可されます。
- **変数の型は宣言時に固定される** –一度 `int` として宣言した変数に `float` を再代入することはできない。
- **ビット演算は `int` のみ** –`float` や `bool` に対してビット演算を適用するとコンパイルエラー。
- **`bool` 以外の型も条件式に使える** –`if` の条件式には `int` (`0 = false`、非 `0 = true`) など `bool` 以外も使用可能。
- **数値リテラルサフィックス** –低レベル型はリテラルサフィックスで指定可能: `42i32`、`255u8`、`3.14f32`、`0xFFu8`、`0b1010u8`。`float` サフィックスを付けた整数リテラル (`42f32`) は `float` 値を生成します。整数サフィックスを付けた浮動小数点リテラル (`3.14i32`) はコンパイルエラーです。範囲外の値 (例: `256u8`、`129i8`) もコンパイルエラーです。
- **低レベル数値型 (`i8`、`i16`、`i32`、`i64`、`u8`、`u16`、`u32`、`u64`、`f32`) は暗黙の変換なし** –低レベル型同士または高レベル型 (`int`、`float`) との混合はコンパイルエラーです。明示的な `as` キャストを使用してください。低レベル整数の `/` 演算子は (Rust と同様に) `float` 除算ではなく整数除算を行います。符号付き型は `SDiv/SRem`、符号なし型は `UDiv/URem` を使用します。
- **符号付き vs 符号なし** –符号付き型 (`i8`、`i16`、`i32`、`i64`) は符号付き比較 (`ICMP_SLT` 等) と算術右シフト (`AShr`) を使用します。符号なし型 (`u8`、`u16`、`u32`、`u64`) は符号なし比較 (`ICMP_ULT` 等) と論理右シフト (`LShr`) を使用します。`>>>` 演算子は符号に関わらず常に論理シフトを行います。
- **低レベル整数のオーバーフローはラップアラウンド** –低レベル整数型の算術演算はオーバーフロー時に 2 の補数ラップ (符号付き) またはモジュラー演算 (符号なし) を使用します。例えば、`i32` の最大値 `2147483647 + 1` は `-2147483648` にラップします。これは C の動作と一致します。オーバーフローが問題になる場合は高レベルの `int` 型 (64 ビット) を使用してください。明示的なオーバーフロー制御には `checked_add/sub/mul` (`Result<T, Error>` を返す)、`saturating_add/sub/mul` (型の境界値にクランプ)、`wrapping_add/sub/mul` (自己文書化的なラッピング) を使用できます。[関数リファレンス](#)を参照。